دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

برنامه‌سازی پیشرفته و کارگاه

مقدمه‌ای بر شی‌گرایی

نگارش:

شهاب گریوانی، یونس کاظمی، آرمان حسینی، نگین موسی یی جو، نوا جغتایی

استاد درس:

دکتر احمدیان

دانشکده ریاضی و علوم کامپیوتر

دانشگاه صنعتی امیرکبیر

بهار ۱۴۰۵

فهرست مطالب

۱ مقدمه‌ای بر شی‌گرایی ۳
۱.۱ مقدمه ۳
۲.۱ تایپ‌ها در جاوا ۳
۳.۱ کلاس‌ها ۵
۱.۳.۱ تعریف کلاس. ۹
۴.۱ ایجاد Object. ۹
۱.۴.۱ Constructor ها ۱۰
۲.۴.۱ Default Constructor ۱۲
۳.۴.۱ کلاس‌ها با چند کانستراکتور ۱۳
۵.۱ field ها. ۱۳
۱.۵.۱ فیلدهای static ۱۵
۶.۱ method ها ۱۷
۱.۶.۱ متغیرهای محلی (local variables) ۱۸
۲.۶.۱ Shadowing ۲۱
۳.۶.۱ متدهای استاتیک ۲۳
۴.۶.۱ Method overloading ۲۶
۷.۱ Reference type ها ۲۷
۱.۷.۱ تفاوت reference type ها و primitive type ها ۲۸
۲.۷.۱ کلاس‌های wrapper برای primitive هاtype ۳۲
۸.۱ Garbage Collection ۳۳
۱.۸.۱ آشنایی با Memory Leak ۳۳
۲.۸.۱ Garbage Collection: یک راه‌حل خوب ۳۴
۳.۸.۱ Garbage Collector حواسش هست! ۳۵
۴.۸.۱ یک نکته در مورد کلاس‌ها ۳۶
۹.۱ Packages ۳۷
۱.۹.۱ نیاز به منظم کردن فایل‌ها. ۳۷
۲.۹.۱ پکیج چیست؟ ۳۷
۳.۹.۱ چطور یک پکیج ایجاد کنیم؟ ۳۸
۴.۹.۱ نام‌گذاری متداول پکیج‌ها (Naming Conventions) ۴۳
۵.۹.۱ کلیدواژه‌ی Import ۴۳
۶.۹.۱ در اهمیت پکیج‌ها. ۴۴
۷.۹.۱ import کردن کل پکیج ۴۵
۸.۹.۱ چند نکته در مورد پکیج‌ها ۴۷
۱۰.۱ چه چیزی یاد گرفتیم؟ ۴۸

فصل ۱ مقدمه‌ای بر شی‌گرایی

۱.۱ مقدمه

در این داکیومنت و داکیومنت‌های بعدی، با شی‌گرایی آشنا می‌شویم. شی‌گرایی یا object-oriented programming (به اختصار OOP)، طرز فکری است برای تقسیم یک برنامهٔ بزرگ، به واحدهای کوچک‌تر به اسم . class خیلی از چیزهایی که قبلا بهتون گفته بودیم «در آینده برایتان توضیح می‌دهیم» را قرار است این‌جا یاد بگیرید!

پیشنهاد می‌کنیم در طول مطالعه‌ی این داکیومنت با تدریس یارها در ارتباط باشید و اگر سوالی داشتید حتما از آن‌ها بپرسید، زیرا این یکی از مهم‌ترین داکیومنت‌های شماست و پایه‌ایست برای اکثر مطالبی که در ادامه‌ی ترم باهم مورد بحث قرار می‌دهیم مانند ارث‌بری، کپسوله‌سازی و مباحث مشابه .

۲.۱ تایپ‌ها در جاوا

تا به اینجا با بسیاری از type ها مثل `int` ، `float` ، `String` ، `JFrame` ، `ArrayList` و ... کار کرده‌اید و با آن‌ها آشنا شده‌اید؛ هر برنامه‌نویسی در جاوا، می‌تواند به راحتی تایپ‌های جدید برای خودش بسازد و به همین خاطر، احتمالا تا به حال میلیون‌ها تایپ در جاوا نوشته شده‌اند. شما نیز در این داک قرار است یاد بگیرید که چطور می‌توانید تایپ‌های جدید بسازید.

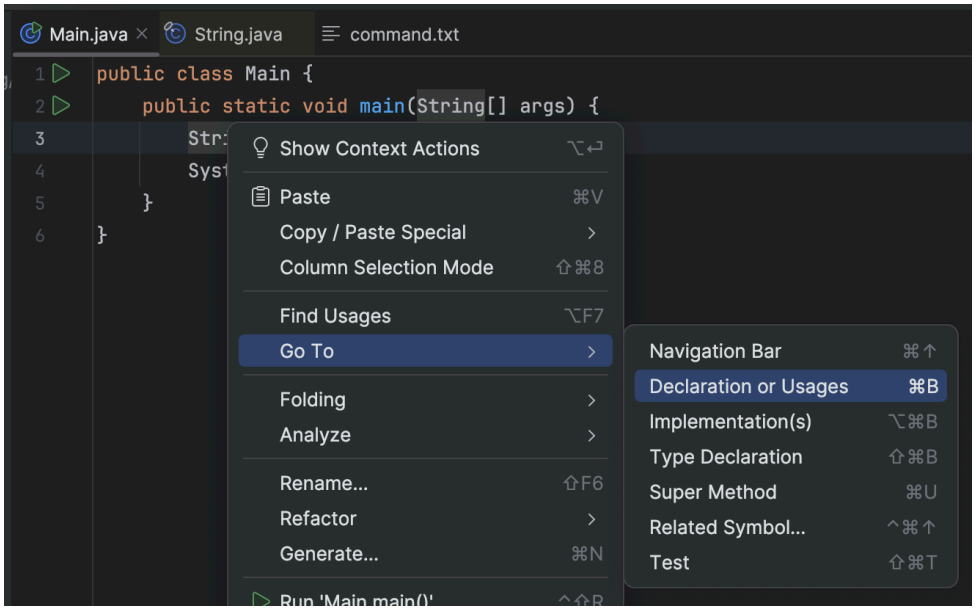
همان‌طور که در جلسه‌ی صفر دیدید، تایپ‌های `int` ، `float` ، `double` ، `long` و `char` ، به اسم «تایپ‌های اولیه» یا primitive types شناخته می‌شوند. فهرست کامل آن‌ها به شکل زیر است:

Type	Definition	Approximate range or precision
`boolean`	Logical Value	`true` or `false`
`char`	16-bit, Unicode character	$64k$ characters
`byte`	8-bit, signed integer	-128 to 127
`short`	16-bit, signed integer	$-32,768$ to $32,676$
`int`	32-bit, signed integer	$-2.1e9$ to $2.1e9$
`long`	64-bit, signed integer	$-9.2e18$ to $9.2e18$
`float`	32-bit, IEEE 754, floating-point value	$6-7$ significant decimal places
`double`	64-bit, IEEE ۷۵۴	15 significant decimal places

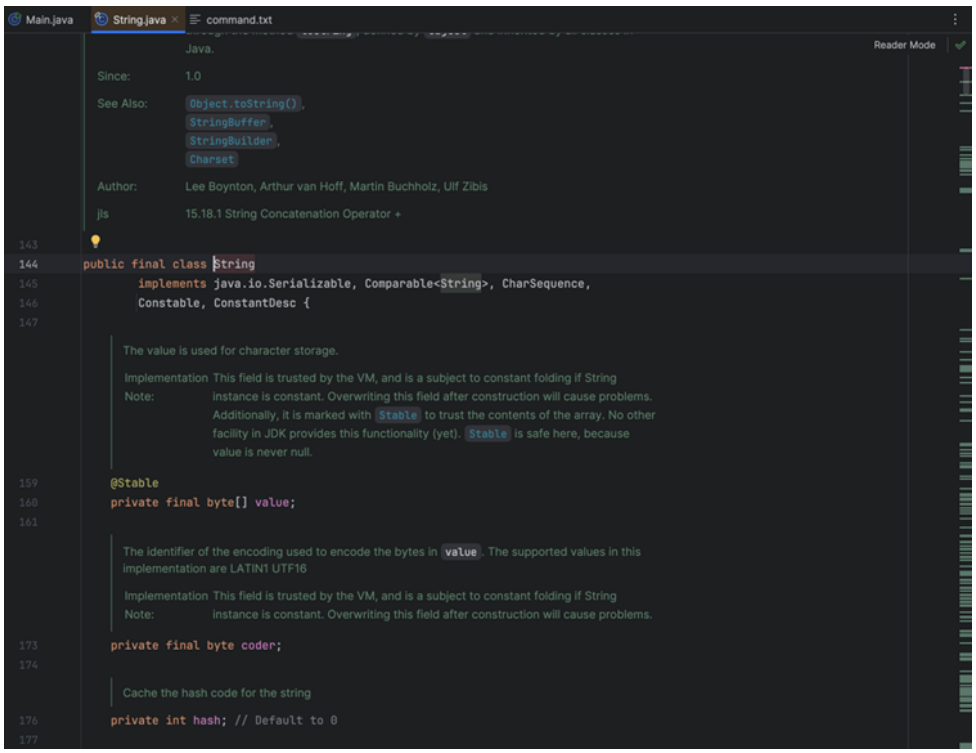
این تایپ‌ها، مانند تیکه‌های کوچک لگو هستند که تایپ‌های پیچیده‌تر مانند `String` ، `JFrame` ، `ArrayList` و هر تایپ دیگری را تشکیل می‌دهند. این تایپ‌های پیچیده‌تر، reference type نام دارند. در ادامه یک مثال از آن‌ها را مورد بررسی قرار می‌دهیم. کد زیر را در IntelliJ کپی کنید:

```
public class Main {
    public static void main(String[] args) {
        String str = "Hello World";
        System.out.println(str);
    }
}
```

موس را روی String ببرید و کلیک راست کنید. از صفحه‌ای که باز می‌شود، منوی Go To و بعد از آن، Declaration or Usages را انتخاب کنید:



صفحه‌ی جدیدی برایتان باز می‌شود که شامل پیاده‌سازی تایپ Stringاست:



احتمالا شما هنوز این کد را کامل نمی‌فهمید، ولی در این کد نشان داده شده است که تایپ String از کنار هم گذاشتن چه تایپ‌های دیگری درست شده است. مثلا در خط ۱۶۰ همین تصویر می‌بینید که String در خودش یک آرایه از byte ها دارد که اسمش value است. پایین‌تر می‌بینید که در خودش، یک int به نام hash دارد. اگر دوست دارید، کمی این کد را بررسی کنید. با آن که تیکه‌هایی از آن برایتان غریبه هستند، بخش‌هایی هم برای شما آشنا به نظر می‌آیند.

۳.۱ کلاس‌ها

کلاس‌ها، به شما اجازه می‌دهند تا type های جدیدی درست کنید. برای شروع، با هم یه کلاس جدید به اسم Student درست می‌کنیم. کد زیر را در IntelliJ بنویسید:

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}

class Student {
    public int age;
    public String name;
    public String studentID;
}
```

در این کد، ما یک class جدید، به نام Student تعریف کرده‌ایم. این کلاس، از چهار متغیر تشکیل شده است:

- متغیر age : متغیری از جنس int که سن دانشجو را نشان می‌دهد.
- متغیر name : متغیرِ String ای که اسم دانشجو را نشان می‌دهد.
- متغیر studentID : متغیر String ای که شماره‌ی دانشجویی شخص را نشان می‌دهد.

متغیرهایی که یک کلاس را تشکیل می‌دهند، field های آن کلاس نام دارند. نگران کلیدواژه‌ی public نباشید، در جلسات آینده مفهوم آن را توضیح خواهیم داد. برای الان، لازم است که آن را پیش از همه‌ی field های کلاس‌هایتان قرار دهید. حالا با کلیدواژهٔ new ، یک دانشجوی جدید به اسم قلی بسازید و فیلدهای آن را مقداردهی کنید:

```
public static void main(String[] args) {
    Student gholi = new Student();

    gholi.age = 20;
    gholi.name = "Gholi";
    gholi.studentID = "40413099";
}
```

سپس مشخصات این دانشجو را چاپ کنید:

```
System.out.println("New student:");
System.out.println("\t+ Name: " + gholi.name);
System.out.println("\t+ StudentID: " + gholi.studentID);
System.out.println("\t+ Age: " + gholi.age);
```

کد خود را اجرا کنید. خروجی زیر را می‌بینید:

```
New student:
    + Name: Gholi
    + StudentID: 40413099
    + Age: 20
```

می‌بینید که مشخصات قلی، به درستی روی صفحه چاپ می‌شود. تبریک! شما اولین type خود را ساختید و یک متغیر از جنس آن درست کردید. می‌خواهیم به این type جدید، چیزهای بیشتری اضافه کنیم. به Student فیلدی به اسم grades از جنس ArrayList<Double> اضافه کنید. این فیلد، نمرات دانشجو در درس‌های مختلف را نشان می‌دهد:

```
public ArrayList<Double> grades;
```

حالا، وقتی دارین فیلدهای مختلف قلی رو توی main مقداردهی می‌کنید، این آرایه هم با یک آرایهٔ خالی مقداردهی کنید:

```
gholi.age = 20;
gholi.name = "Gholi";
gholi.studentID = "40413099";
gholi.grades = new ArrayList<Double>();
```

بعد از این کار، چند نمره‌ی رندوم به قلی بدهید:

```
gholi.grades.add(20.0);
gholi.grades.add(17.0);
gholi.grades.add(18.0);
gholi.grades.add(0.0);
```

حالا، جایی که دارید مشخصات قلی را چاپ می‌کنید، نمرات را هم چاپ کنید:

```
System.out.println("New student:");
System.out.println("\t+ Name: " + gholi.name);
System.out.println("\t+ StudentID: " + gholi.studentID);
System.out.println("\t+ Age: " + gholi.age);
System.out.print("\t+ Grades: ");
for (var grade: gholi.grades) {
    System.out.print(grade + ", ");
}
```

کدتان را اجرا کنید. خروجی شما باید به این شکل باشد:

```
New student:
    + Name: Gholi
    + StudentID: 40413099
    + Age: 20
    + Grades: 20.0, 17.0, 18.0, 0.0,
```

می‌خواهیم به Student ، قابلیت محاسبه‌ی معدل هم بدهیم. برای این کار، متدی جدید به اسم getAverageGrade در Student تعریف می‌کنیم:

```
class Student {
    public int age;
    public String name;
    public String studentID;
    public ArrayList<Double> grades;

    public double getAverageGrade() {
        if (grades.size() == 0) {
            return 0;
        }

        double gradeSum = 0;
        for (double grade: grades) {
            gradeSum += grade;
        }

        return gradeSum / grades.size();
    }
}
```

جایی از کدتان که در آن مشخصات gholi را چاپ می‌کنید، خطوط زیر را اضافه کنید:

```
System.out.println();
System.out.println("\t+ Average Grade: " + gholi.getAverageGrade());
```

کدتان را دوباره اجرا کنید. خروجی‌ای مثل زیر می‌بینید:

```
New student:
    + Name: Gholi
    + StudentID: 40413099
    + Age: 20
    + Grades: 20.0, 17.0, 18.0, 0.0,
    + Average Grade: 13.75
```

لحظه‌ای به خود تابع main توجه کنید. می‌بینید که خود آن نیز در کلاسی به اسم Mainاست! شما در تمام این مدت داشتید کلاس Main را تعریف می‌کردید، بدون آن که خبر داشته باشید:

```
public class Main {
    public static void main(String[] args) {
        // Your code here
```

```
    }
}
```

شما حتی می‌توانید از این کلاس هم یک متغیر درست کنید:

```
public class Main {
    public static void main(String[] args) {
        Main a = new Main();
    }
}
```

البته که متغیر ساختن از جنس Main خیلی کار خوبی نیست! ولی جالب است که تا همین الآن هم شما از class ها استفاده می‌کردید، بدون آن که بدانید. =)))
در ادامه، کمی رسمی‌تر و قدم به قدم‌تر کلاس‌ها را بررسی می کنیم.

۱.۳.۱ تعریف کلاس

تعریف کلاس، کار سختی نیست، فقط از کلیدواژهٔ class استفاده کنید و اسم کلاستان را بنویسید:

```
class Student {
    // Everything an student can do
}
```

هر چیزی که بین دو براکت قرار گیرد، متعلق به آن کلاس است. کلاس‌ها، به شما اجازه می‌دهند که کدهای خیلی خیلی بزرگ را به تیکه‌های کوچیک‌تر تقسیم کنید و با این کار، برنامه‌های منظم‌تر و بهتری داشته باشید.
یادآوری می‌کنیم که type هایی که به وسیلهٔ class ها تعریف می‌شوند، همگی از reference type ها محسوب می‌شوند.

۴.۱ ایجاد Object

به متغیرهایی که از جنس یک reference type هستند، object یا instance می‌گویند.در کد زیر، قلی و ممد و سلطان همگی object هایی از جنس Student ان:

```
public static void main(String[] args) {
    Student gholi;
    Student mamad;
    Student soltan;
}
```

تلاش کنید این کد را اجرا کنید. می‌بینید که طبیعتا بدون هیچ مشکلی اجرا می‌شود. شما توی این کد سه object جدید تعریف کردید و هیچ‌جایی آن‌ها را مقداردهی نکردید و جایی هم از آن‌ها استفاده نشده است.

برای این که آن‌ها را مقداردهی کنید، از کلیدواژهٔ new استفاده کنید:

```
Student gholi = new Student();
Student mamad = new Student();
Student soltan = new Student();
```

حالا فیلدهای ان‌ها را مقداردهی کنید:

```
gholi.age = 21;
gholi.name = "Gholi";
gholi.studentID = "40513089";
gholi.grades = new ArrayList<>();

mamad.age = 25;
mamad.name = "Mamad";
mamad.studentID = "40513090";
mamad.grades = new ArrayList<>();

soltan.age = 103;
soltan.name = "Soltan";
soltan.studentID = "40513091";
soltan.grades = new ArrayList<>();
```

همانطور که می‌بینید ما در این کد به اجبار کد های تکراری زیادی زده‌ایم. کدی که مربوط به بخش مقداردهی field های یک object است سه بار تکرار می‌شود پس بهتر است این کدها را به یک متد تبدیل کنیم. Constructor ها، متدهایی هستند که دقیقا برای همین کار، به کمک ما می‌آیند.

۱.۴.۱ Constructor ها

به کلاس Student ، متد زیر را اضافه کنید:

```
public Student(int newStudentAge, String newStudentName, String newStudentID)
↪ {
    age = newStudentAge;
    name = newStudentName;
    studentID = newStudentID;
```

```
    grades = new ArrayList<>();
}
```

می‌بینید که ظاهر این متد، کمی متفاوت است! نوع خروجی‌اش مشخص نشده و جایی از کلیدواژه‌ی return استفاده نکرده است. به این متد، Constructor می‌گویند و وظیفه‌ی آن، مقداردهی یک object جدید از کلاس Student است. در constructor بالا، ما برای ساخت یه Student جدید، سن، اسم و شماره دانشجویی‌اش را ورودی گرفته‌ایم و با استفاده از آن‌ها فیلدهای Student را برای object جدیدمان مقداردهی کرده‌ایم.

برای این که از این constructor استفاده کنید، کد توی main را با کد زیر جایگزین کنید:

```
public static void main(String[] args) {
    Student gholi = new Student(21, "Gholi", "40513089");
    Student mamad = new Student(25, "Mamad", "40513090");
    Student soltan = new Student(103, "Soltan", "40513091");
}
```

می‌بینید که با استفاده از یه constructor خوب، چقدر کد main کوتاه‌تر و تمیزتر شد! حالا متد زیر را به Student اضافه کنید:

```
public void printInfo() {
    System.out.println("Student info:");
    System.out.println("\t+ Name: " + name);
    System.out.println("\t+ StudentID: " + studentID);
    System.out.println("\t+ Age: " + age);
    System.out.print("\t+ Grades: ");
    for (var grade: grades) {
        System.out.print(grade + ", ");
    }
    System.out.println();
    System.out.println("\t+ Average Grade: " + getAverageGrade());
}
```

و توی main ، این متد را صدا بزنید تا اطلاعات ممد و قلی و سلطان چاپ شود:

```
gholi.printInfo();
mamad.printInfo();
soltan.printInfo();
```

خروجی‌ای مثل زیر می‌بینید:

```
Student info:
    + Name: Gholi
    + StudentID: 40513089
    + Age: 21
    + Grades:
    + Average Grade: 0.0
Student info:
    + Name: Mamad
    + StudentID: 40513090
    + Age: 25
    + Grades:
    + Average Grade: 0.0
Student info:
    + Name: Soltan
    + StudentID: 40513091
    + Age: 103
    + Grades:
    + Average Grade: 0.0
```

می‌بینیم که constructor ها کار کردند و کد ما هم، کوتاه و تمیز شد!

۲.۴.۱ Default Constructor

پیش از آن که ما برای کلاس Student کانستراکتور تعریف کنیم، می‌توانستیم با استفاده از کلیدواژهٔ new از آن object بسازیم:

```
Student gholi = new Student();
```

خب، کمی بهش فکر کنید. چگونه این اتفاق افتاد؟ ما که constructor ای برای Student نداشتیم! با نوشتن این کد، دقیقا چه constructor ای صدا زده می‌شود؟

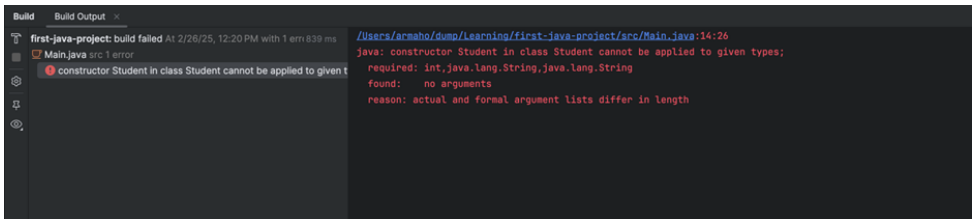
وقتی یک کلاس constructor نداشته باشد، جاوا به طور پیش‌فرض یک constructor خالی برایش می‌نویسد. این constructor به شکل زیر است:

```
public Student() {}
```

در این کانستراکتور هیچ اتفاقی نمی‌افتد و هیچ field ای مقداردهی نمی‌شود. صرفاً وجود دارد تا شما بتوانید به راحتی با استفاده از new Student() ، آبجکت‌های جدیدی از این کلاس بسازید. به محض آن که شما اولین constructor کلاس Student را بنویسید، دیگر نمی‌توانید از default constructor جاوا استفاده کنید. در کد main فعلیتان این خط را بنویسید و آن را اجرا کنید:

```
Student pedram = new Student();
```

خطایی مانند زیر می‌گیرید:



از آن‌جایی که کلاس student ، الآن یک constructor دارد، جاوا برای شما کانستراکتور دیفالت را درست نمی‌کند.

۳.۴.۱ کلاس‌ها با چند کانستراکتور

شما می‌توانید برای رفع مشکلی که بالاتر به آن برخورد کردید، خودتان یک کانستراکتور خالی برای Student بنویسید:

```
public Student() {}
```

دقت کنید که لازم نیست کانستراکتور قبلی خود را پاک کنید! هر کلاس، می‌تواند چندین کانستراکتور داشته باشد. شما می‌توانید این دو کانستراکتور را در کنار هم بنویسید و کدتان همچنان کار می‌کند.

۵.۱ field ها

متغیر های یک کلاس، یا field ها، ویژگی‌های آن کلاس را نشان می‌دهند. شما تا الآن تعداد خوبی field برای کلاس student تعریف کردید و با آن‌ها آشنا شدید:

```
class Student {
    public int age;
    public String name;
    public String studentID;
    public ArrayList<Double> grades;

    // Other things happening in the Student class

}
```

شما حتی می‌توانید در یک کلاس، field هایی از جنس همان کلاس تعریف کنید! مثلاً، می‌توانید فیلد friend را برای Student ، از جنس خود Student تعریف کنید:

```
class Student {
    // Other fields

    public Student friend;

    // Other things happening in the Student class

}
```

حالا، یک student جدید به اسم شهرام توی کدتان تعریف کنید:

```
var shahram = new Student();
```

برای الآن، بدون این که فیلدهای شهرام را مقداردهی کنید، سعی کنید تا اطلاعات او را چاپ کنید:

```
System.out.println("Shahram: ");
System.out.println("\t+ Name: " + shahram.name);
System.out.println("\t+ Age: " + shahram.age);
System.out.println("\t+ StudentID: " + shahram.studentID);
```

کدتان را اجرا کنید، خروجی زیر را برای شهرام می‌بینید:

```
Shahram:
    + Name: null
    + Age: 0
    + StudentID: null
```

می‌بینید که علی رغم آن که شما به شهرام اسم و سن و شماره دانشجویی ندادید، خود جاوا یک سری مقدار به آن‌ها اختصاص داده است. مقدار دیفالت جاوا برای فیلدهایی که مقداردهی نشده‌اند، به این شکل است:

- **متغیرهای عددی (مثل int ، float و امثال اون‌ها):** مقدار 0 را به خود می‌گیرند.
- **متغیرهای char :** مقدار '\0' را به خود می‌گیرند.
- **متغیرهای boolean :** مقدار false به خود می‌گیرند.
- **متغیرهایی از جنس reference type :** مقدار null به خود می‌گیرند.

کلیدواژه‌ی `null` ، یکی از کلیدواژه‌های خاص جاواست که در واقع یک مقدار نیست، بلکه نشان‌دهنده‌ی آن است که یک متغیر از جنس reference type ، هنوز مقداری به خودش نگرفته است. اگر توی کد قبلیمان، خط زیر را بنویسیم:

```
if (shahram.studentID == null) {
    System.out.println("Shahram does not have a studentID");
}
```

می‌بینید که پیام زیر چاپ می‌شود:

```
Shahram does not have a studentID
```

جلوتر، با این کلیدواژه بهتر آشنا می‌شویم.

۱.۵.۱ فیلدهای static

بعضی فیلدها(ویژگی‌ها)، متعلق به هیچ object ای نیستند، ولی بی‌ربط به خود `class` هم نیستند. مثلا در کلاس `Student` ویژگیِ «تعداد کل دانشجوها» متعلق به هیچ کدام از قلی، ممد یا سلطان نیست، ولی به کلاس `Student` ربط دارد.
به این ویژگی‌ها، ویژگی‌های `static` گویند. آن‌ها به خود class مرتبط هستند و بین تمام instance های آن class مشترکند. فیلد`static` زیر را برای دانشجوها تعریف کنید:

```
class Student {
    public static int totalNumberOfStudents = 0;

    // other stuff

}
```

سپس در همه‌ی constructor هایی که برای `Student` نوشته‌اید، به مقدار آن یکی اضافه کنید. با این کار، با ساخت هر دانشجو، تعداد کل دانشجوها یکی زیاد می‌شود:

```
public Student() {
    totalNumberOfStudents++;
}

public Student(int newStudentAge, String newStudentName, String newStudentID)
↪ {
    age = newStudentAge;
    name = newStudentName;
```

```
    studentID = newStudentID;
    grades = new ArrayList<>();

    totalNumberOfStudents++;
}
```

حالا، کد زیر را در main بنویسید:

```
public static void main(String[] args) {
    var gholi = new Student();
    System.out.println("Current number of students: " +
↪   Student.totalNumberOfStudents);

    var mamad = new Student();
    System.out.println("Current number of students: " +
↪   Student.totalNumberOfStudents);

    var javad = new Student();
    System.out.println("Current number of students: " +
↪   Student.totalNumberOfStudents);
}
```

همچین خروجی‌ای می‌بینید:

```
Current number of students: 1
Current number of students: 2
Current number of students: 3
```

می‌بینید که ما برای دسترسی به totalNumberOfStudents ، از خود کلاس Student استفاده کرده‌ایم. می‌توانستید با کد زیر، از هر کدام از instance های student نیز به آن دسترسی پیدا کنید، ولی کار چندان خوبی نیست:

```
System.out.println("Current number of students: " +
↪   gholi.totalNumberOfStudents);
```

۶.۱ method ها

تا اینجای کار، با کلاس‌های نسبتاً ساده‌ای سر و کار داشتید. اما جاهای مختلف به «رفتار کلاس» یا این ایده که کلاس ما کاری انجام بدهد اشاره کردیم، اینجاست که متدها وارد عمل می‌شوند: به طور کلی زمانی که بخواهید در کدتان تصمیمی بگیرید و یا عملیات منطقی یا به طور کلی کاری انجام بدهید، باید از متدها استفاده کنید. متدها آنقدر مهم هستند که حتی در اولین مواجه‌تان با جاوا از متد main استفاده کردید و داخل آن کد خود را نوشتید! در این بخش قرار است دقیق‌تر و کامل‌تر با متدها آشنا بشوید. کد زیر، یک مثال ساده از یک متد است:

```
public class Refrigerator {
    int numberOfBananas;

    public void getBananas(int n) {
        boolean enoughBananas = numberOfBananas >= n;
        if (enoughBananas) {
            numberOfBananas -= n;
            System.out.println(
                    "You took " + n + " bananas out of your fridge!"
            );
        } else {
            System.out.println(
                    "You don't have that many bananas in your fridge!"
            );
        }
    }
}
```

در این مثال، یک کلاس Refrigerator داریم که یک فیلد از نوع int به نام numberOfBananas دارد. همچنین یک متد دارد که نوع خروجی‌اش void است (خروجی ندارد) و یک ورودی (argument) از نوع int دارد. با استفاده از این متد می‌توانید از توی یخچالتان موز بردارید! حالا خودتان متدی اضافه کنید که با آن بتوانید توی یخچالتان موز بگذارید. متدتان احتمالاً چیزی شبیه به این می‌شود:

```
public void putBananas(int n) {
    numberOfBananas += n;
    System.out.println("You put " + n + " bananas in your fridge!");
}
```

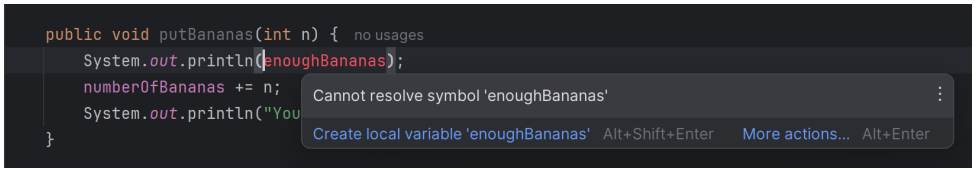
اینجا، متدمان تعداد مشخص و ثابتی ورودی دارد (یکی)؛ اما می‌توانید متدهایی تعریف کنید که تعداد ورودی‌هایشان ثابت نباشد. در این درس نیازی به یادگیری این کار ندارید اما در صورت علاقه می‌توانید کلمه‌ی "varargs" را جستجو کنید یا از این لینک راجع به آن بخوانید.

۱.۶.۱ متغیرهای محلی (local variables)

متد getBananas() که در مثال بخش قبل تعریف کردیم، قبل از هر چیزی چک می‌کند که توی یخچال به اندازه‌ی کافی موز وجود داشته باشد و این را در یک متغیر محلی به اسم enoughBananas ذخیره می‌کند. متغیرهای محلی موقتی هستند و فقط توی همان متدی که تعریف شده‌اند قابل استفاده‌اند. این متغیرها وقتی متد صدا زده می‌شود ساخته می‌شوند و معمولاً بعد از تمام شدن متد از بین می‌روند. همچنین از بیرون متد نمی‌توانید به آنها دسترسی داشته باشید. برای آن که خودتان ببینید، در همین کلاس Refrigerator سعی کنید در متد putBananas از متغیر enoughBananas استفاده کنید؛ برای مثال:

```
public void putBananas(int n) {
    System.out.println(enoughBananas);
    numberOfBananas += n;
    System.out.println("You put " + n + " bananas in your fridge!");
}
```

احتمالاً می‌بینید که enoughBananas قرمز شده است. موس را ببرید روی آن ؛ با همچین چیزی مواجه می‌شوید:

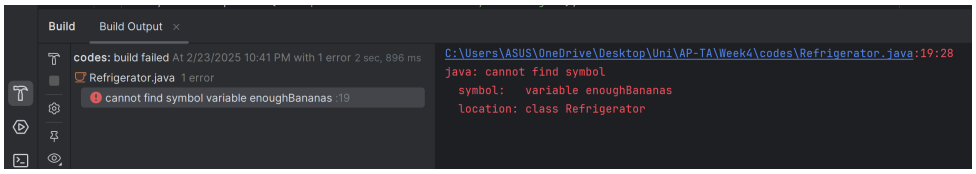


اینجا Intellij دارد به شما می‌گوید که نمی‌تواند enoughBananas را پیدا کند! دلیلش هم همانطور که گفتیم آن است که enoughBananas داخل متد getBananas تعریف شده و مربوط به همان متد است و داخل putBananas همچین متغیری وجود ندارد! حالا یک متد main خالی داخل کلاستان بنویسید و سعی کنید اجرایش کنید:

```
public class Refrigerator {
    int numberOfBananas;

    public void getBananas(int n) {
        boolean enoughBananas = numberOfBananas >= n;
        if (enoughBananas) {
            numberOfBananas -= n;
            System.out.println(
                "You took " + n + " bananas out of your fridge!"
            );
            } else {
            System.out.println(
                "You don't have that many bananas in your fridge!"
```

```
            );
        }
    }

    public void putBananas(int n) {
        System.out.println(enoughBananas);
        numberOfBananas += n;
        System.out.println("You put " + n + " bananas in your fridge!");
    }

    public static void main(String[] args) {
    }
}
```

کدتان کامپایل نمی‌شود. با همچین چیزی مواجه می‌شوید:



اینجا جاوا نتوانسته کد شما را کامپایل کند و به شما می‌گوید که نمی‌تواند enoughBananas را پیدا کند. باید هشدارهای Intellij را جدی می‌گرفتید!

ورودی‌های یک متد هم جزو متغیرهای محلی آن متد حساب می‌شوند، با این تفاوت که مقدار اولیه‌شان موقعی که متد صدا زده می‌شود از طرف کسی که متد را فراخوانی کرده است، مشخص می‌شود.

مقداردهی اولیه به متغیرهای محلی

بر خلاف فیلدهای آبجکت که اگر مقداردهیشان نکنید، جاوا برای آنها مقدار پیش‌فرضی قرار می‌دهد، متغیرهای محلی را باید قبل از استفاده کردن مقداردهی کنید وگرنه خطای کامپایل می‌گیرید:

```
public class SomeClass {
    // instance variables always get default values if
    // you don't initialize them
    int foo;

    void myMethod() {
    // local variables do not get default values
        int bar;
        foo += 1; // This is ok, foo has the value 0
        bar += 1; // compile-time error, bar is uninitialized
```

```
    }

    public static void main(String[] args) {
        SomeClass something = new SomeClass();
        something.myMethod();
    }
}
```

سعی کنید کد بالا را اجرا کنید. می‌بینید که کدتان حتی کامپایل هم نمی‌شود! باید اول bar را مقداردهی کنید:

```
bar = 99; // This is ok, we're setting bar's initial value
bar += 1; // Now this calculation is ok
```

البته دقت کنید که لازم نیست حتماً موقع تعریف کردن یک متغیر آن را مقداردهی کنید؛ بلکه قبل از آن که از آن استفاده کنید باید مقداردهی شده باشد. موضوع وقتی پیچیده‌تر می‌شود که مقداردهی را داخل یک شرط انجام بدهید:

```
void myMethod() {
    int bar;
    if (someCondition) {
        bar = 42;
    }
    bar += 1; // Still a compile-time error, bar may not be initialized
}
```

در این مثال، bar فقط در صورتی مقداردهی می‌شود که شرط someCondition برقرار باشد. یعنی همچنان ممکن است که قبل از خط bar += 1 ، متغیر bar مقداردهی نشده باشد. کامپایلر به شما اجازه نمی‌دهد همچین کاری بکنید و این کد هم خطای کامپایل می‌دهد.

برای حل این مشکل، چند راه حل وجود دارد. می‌توانید متغیر را قبل از بلوک شرط مقداردهی کنید، یا استفاده‌ای که از متغیر می‌کنید را هم داخل بلوک شرط ببرید، یا می‌توانید با توجه به برنامه‌ای که دارید می‌نویسید، به نحوی مطمئن بشید که متغیر قبل از مقداردهی استفاده نمی‌شود. برای مثال، توی کد بالا می‌توانید bar را هم در بلوک if و هم در بلوک else مقداردهی کنید یا در صورتی که someCondition برقرار نبود، متد را تمام کنید و return کنید:

```
void myMethod() {
    int bar;
    if (someCondition) {
        bar = 42;
```

```
    } else {
        return;
    }
    bar += 1; // This is ok!
}
```

در این کد، یا bar مقداردهی می‌شود و بعد از آن استفاده می‌شود، یا کلا متد قطع می‌شود و return می‌کند. جاوا این را ازتون می‌پذیره!

حالا چرا اصلاً جاوا انقدر روی این موضوع حساس است؟ یکی از متداول‌ترین مشکلاتی که در زبان‌هایی مثل C و C++ به وجود می‌آید آن است که یادتان می‌رود متغیری را مقداردهی کنید. توی این زبان‌ها، متغیرهای مقداردهی نشده، مقادیر ظاهراً رندومی اختیار می‌کنند و این می‌تواند دردسرساز شود و باعث سخت‌تر شدن پروسه‌ی دیباگ برنامه‌ها بشود. جاوا با مجبور کردن شما به مقداردهی به متغیرها، باعث جلوگیری از این مشکل می‌شود.

۲.۶.۱ Shadowing

وقتی که نام یک متغیر محلی یا یک ورودی متد با نام یکی از فیلد های کلاسمان یکی باشد، آن متغیر محلی، اصطلاحا روی آن فیلد "سایه می‌اندازد" و جلوی دسترسی ما به آن فیلد را می‌گیرد، زیرا در این حالت جاوا نمی‌داند که ما هنگام صدا زدن آن نام، قصد استفاده از متغیر محلی را داریم یا فیلد متد را، پس به طور پیش‌فرض استفاده از متغیر محلی را اولویت می‌دهد. شاید فکر کنید این مشکل به ندرت پیش می‌آید، ولی shadowing اتفاق نسبتاً متداولی است، مخصوصا وقتی که متغیرهایمان اسم‌های متداولی داشته باشند. مثال زیر را ببینید:

```
public class Car {
    double x;
    double y;

    public void moveTo(double x, double y) {
        System.out.println("The car is moving to " + x + ", " + y);
    }
}
```

اینجا ما یک کلاس به نام Car داریم که فعلاً فقط دو فیلد برای مختصات دارد (x و y). یک متد moveTo برای آن تعریف کرده‌ایم که قرار است ماشین را برای ما حرکت دهد. همانگونه که می‌بینید، فعلاً متد moveTo فقط دارد x و y را چاپ می‌کند. اما این x و y ، کدام x و y هستند؟ اگر مثلاً مختصات ماشین ما (3,4) باشد و ما متد moveTo را روی ماشین صدا بزنیم و به آن مقادیر (6,7) را بدهیم، چه چیزی چاپ می‌شود؟ خودتان امتحان کنید! توی همین کلاس یک متد main بنویسید، در آن یک آبجکت جدید از Car بسازید، به آن x و y بدهید و متد moveTo را رویش صدا بزنید.

همانطور که می‌بینید، moveTo همان مقادیری را چاپ می‌کند که به آن ورودی دادیم؛ اما اگر بخواهیم ماشین را حرکت بدهیم، باید بتوانیم مختصاتش را تغییر دهیم، ولی چطور می‌توانیم به

فیلدهای x و y که مربوط به آبجکتمان هستند دسترسی پیدا کنیم؟

this

هروقت نیاز دارید که صریحاً به آبجکتی که تویش هستیم یا یکی از اعضای آن اشاره کنید، می‌توانید از کلیدواژه‌ی this استفاده کنید. بیاید دوباره با مثال moveTo ببینیم:

```
public class Car {
    double x;
    double y;
    double gas;

    public void moveTo(double x, double y) {
        double distance = Math.sqrt(
            (this.x - x) * (this.x - x) + (this.y - y) * (this.y - y)
        );
        if (5 * distance > gas) {
            System.out.println("Not enough gas!");
        } else {
            this.x = x;
            this.y = y;
            gas -= 5 * distance;
            System.out.println("The car is moving to " + x + ", " + y);
        }
    }
}
```

اینجا، ابتدا فاصله‌ای که قرار است طی بشود را حساب کرده و توی متغیر محلی distance ریختیم. همانطورکه می‌بینید، برای دسترسی به x و y مربوط به آبجکت (مختصات فعلی ماشین)، از this.x و this.y استفاده کردیم. this در واقع به همان آبجکتی که تویش هستیم اشاره می‌کند.

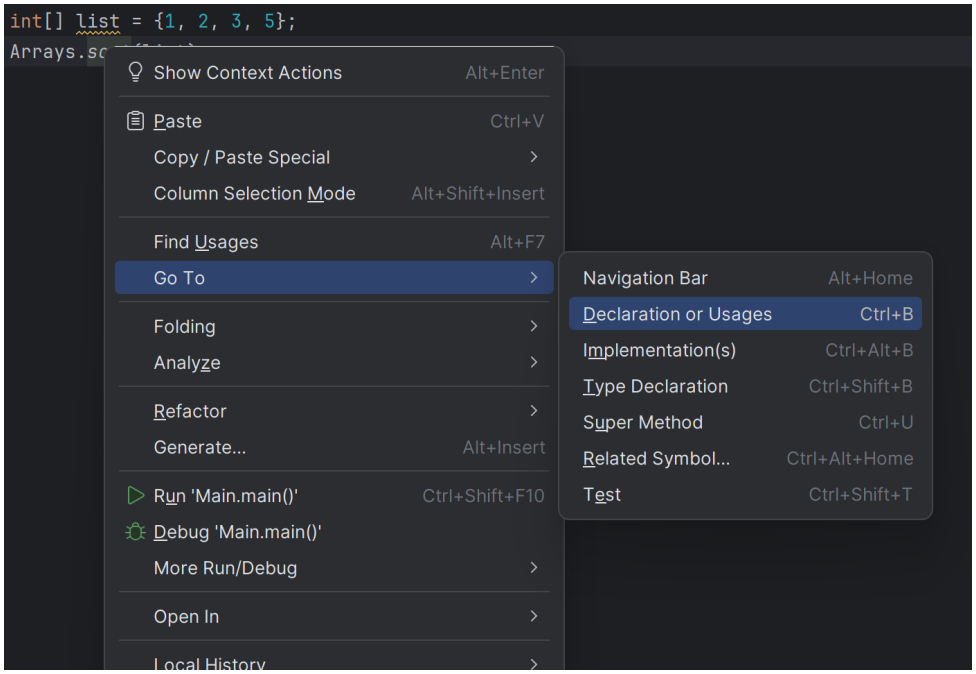
دقت کنید که یک فیلد gas نیز به Car اضافه کردیم که قرار است مقدار بنزین ماشین را نشان بدهد. در ادامه‌ی متد، ابتدا مطمئن می‌شویم که ماشین به اندازه‌ی کافی بنزین دارد و سپس ماشین را حرکت می‌دهیم. می‌بینید که برای دسترسی به gas از this استفاده نکردیم؛ چرا که اشاره به آبجکتی که در آن هستیم به طور ضمنی برقرار است و gas و this.gas اینجا یک چیز هستند.

استفاده از this برای دسترسی به فیلدهایی که روی آنها سایه افتاده، روش مرسومی است و باعث می‌شو بتوانیم از اسم‌های متداولی که برای متغیرهای مختلف وجود دارد (مثلاً x و y برای مختصات) استفاده کنیم و لازم نباشد هر بار دنبال اسمی جدید برای متغیرهایمان باشیم. علاوه بر این، هر جای دیگری که بخواهید به آبجکتی که داخلش هستید اشاره کنید، می‌توانید از this استفاده کنید. مثلاً وقتی بخواهید خود آبجکت را به‌عنوان ورودی به یک متد بفرستید.

۳.۶.۱ متدهای استاتیک

متدهای استاتیک (static methods) ، مثل فیلدهای استاتیک، به خود کلاس تعلق دارند، نه به آبجکت‌های مستقلی که ما از روی آن کلاس می‌سازیم. اما این یعنی چه؟ اول از همه، متدهای استاتیک خارج از آبجکت‌ها وجود دارند و برای صدا زدنشان لازم نیست آبجکتی وجود داشته باشد؛ شما می‌توانید اسم کلاس را بنویسید و با عملگر نقطه متدهای استاتیک را صدا بزنید. قبلاً از متدهای استاتیک زیاد استفاده کرده‌اید، مثلاً برای مرتب کردن آرایه‌ها از `Arrays.sort()` استفاده می‌کردید در حالی که آبجکتی از کلاس `Arrays` نساخته بودید و مستقیماً متد `sort` را روی کلاس `Arrays` صدا می‌زدید؛ این کار قابل انجام است چون `sort` یک متد استاتیک است.

دوباره از `Arrays.sort()` استفاده کنید یا جایی که آن را نوشته‌اید را بیاورید، بعد روی آن راست کلیک کنید و گزینه‌ی Go To و بعد Declaration or Usages را انتخاب کنید:



با همچین کدی مواجه می‌شوید:

```
@Contract(mutates = "param1")
public static void sort( @NotNull int[] a) {
    DualPivotQuicksort.sort(a, parallelism: 0, low: 0, a.length);
}
```

همان‌طور که می‌بینید، پشت این متد از کلیدواژه‌ی `static` استفاده شده. این نشان می‌دهد که این متد، یک متد استاتیک است. حال که می‌دانید متدهای استاتیک چگونه تعریف می‌شوند، بیایید برای کلاس `Car` که تا الان داشتیم یک متد استاتیک تعریف کنیم:

```
public class Car {
    public static final int SUV = 0;
    public static final int SEDAN = 1;
    public static final int HATCHBACK = 2;
```

```
    double x;
    double y;
    double gas;
    int model;

    public static String[] getCarModels() {
        return new String[]{"SUV", "SEDAN", "HATCHBACK"};
    }
    // ...
```

اینجا، اول از همه یک فیلد جدید به ماشین‌هایمان اضافه کردیم به اسم `model` که مدل ماشین ما را نشان می‌دهد: ماشین ما می‌تواند شاسی‌بلند (`model = 0`)، سواری (`model = 1`) یا هاچ‌بک (`model = 2`) باشد. برای راحتی، مدل‌های مختلف ماشین را به صورت فیلدهای `static final` تعریف کردیم. حالا فرض کنید به اسم این مدل‌ها به صورت `String` نیاز داشته باشیم، می‌توانیم مثل بالا یک متد استاتیک تعریف کنیم که این اطلاعات را به ما بدهد. دقت کنید که مدل‌های مختلف ماشین‌ها هیچ ارتباطی به یک ماشین خاص یا یک آبجکت مشخص از نوع `Car` ندارند و به‌طور کلی برای همه ماشین‌ها یکسان هستند. به همین خاطر، استفاده از فیلدها و متدهای استاتیک بهترین انتخاب است.

اصلی‌ترین کاربرد متدهای استاتیک، برای تعریف متدهای کمکی است؛ متدهایی که یا مستقل از آبجکت‌ها کار می‌کنند، یا روی آبجکت‌هایی که از آن کلاس (یا حتی کلاس‌های دیگر) می‌سازیم، عملی انجام می‌دهند و منطقشان به یک instance خاص تعلق ندارد.

حالا بیاید یک متد استاتیک دیگر برای `Car` بنویسیم:

```
public void printModelsCount(ArrayList<Car> list) {
    int suvCount = 0;
    int sedanCount = 0;
    int hatchbackCount = 0;
    for (Car car : list) {
        switch (car.model) {
            case 0:
                suvCount++;
                break;
            case 1:
                sedanCount++;
                break;
            case 2:
                hatchbackCount++;
                break;
        }
```

```
    }
    System.out.println("SUV: " + suvCount);
    System.out.println("SEDAN: " + sedanCount);
    System.out.println("HATCHBACK: " + hatchbackCount);
}
```

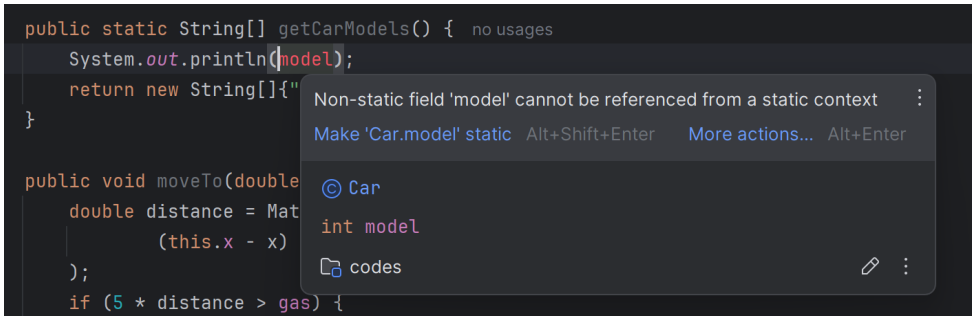
این متد، یک ArrayList از ماشین‌ها می‌گیرد، تعداد مدل‌های مختلف ماشین‌ها را می‌شمارد و چاپ می‌کند. همان‌طور که می‌بینید کارکرد این متد هیچ ربطی به یک instance خاص از Car ندارد و به همین دلیل استاتیک تعریف می‌شود.

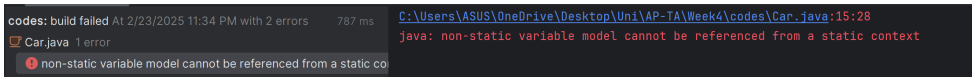
مثال خوب دیگری برای کاربرد متدهای استاتیک، کلاس Math است. این کلاس قرار است مجموعه‌ای از عملیات‌های ریاضی باشد؛ به همین دلیل تمام متدهای کلاس Math استاتیک هستند. البته Math یک مرحله فراتر می‌رود، شما نمی‌توانید اصلاً آبجکتی از Math بسازید! اصلاً این که یک آبجکت از روی Math بسازید، معنی ندارد و نیازی به آن نیست. این کلاس صرفاً قرار است که مجموعه‌ای از متدها و متغیرها برای انجام عملیات ریاضی باشد. شما چند "ریاضی" مختلف ندارید که بخواهید instance های مختلفی از Math بسازید!

حالا سعی کنید در یکی از متدهای استاتیک Car از یکی از فیلدها یا متدهای غیر استاتیک Car استفاده کنید. مثلاً:

```
public static String[] getCarModels() {
    System.out.println(model);
    return new String[]{"SUV", "SEDAN", "HATCHBACK"};
}
```

اگر موستان را ببرید روی model یا سعی کنید کلاس Car را کامپایل کنید و جایی از آن استفاده کنید، با همچین خطاهایی مواجه می‌شوید:





همونطور که میبینید، جاوا دارد به شما می‌گوید که نمی‌توانید یک فیلد غیر استاتیک مثل model را در یک متد استاتیک استفاده کنید. شما می‌توانید متد getCarModels را بدون داشتن هیچ

آبجکتی صدا بزنید؛ پس فیلد `model` ی که سعی دارید از آن استفاده کنید، مربوط به کدام آبجکت است؟ از آن‌جایی که متدهای استاتیک مربوط به کلاس هستند و از آبجکت‌ها جدا هستند، طبیعی است که نمی‌توانند به فیلدها و متدهای عادی که مربوط به هر آبجکت هستند دسترسی داشته باشند و فقط به متد ها و متغیر های استاتیک دسترسی دارند.

۴.۶.۱ Method overloading

قابلیت Method overloading ، این قابلیت است که شما چند متد را با یک اسم، ولی با نوع و تعداد ورودی متفاوت در یک کلاس تعریف کنید؛ وقتی که متد را صدا می‌زنید، کامپایلر با توجه به نوع ورودی، متد درست را انتخاب می‌کند و اجرا می‌کند.

قابلیت Method overloading ، بسیار قدرتمند و پرکاربرد است. ایده اصلی این است که متدهایی درست کنیم که روی ورودی‌های مختلف، کارهای یکسانی انجام بدهند. با این کار می‌توانید این توهم را ایجاد کنید که یک متد می‌تواند روی انواع مختلفی از ورودی‌ها کار کند. متد `println()` که از اولین جلسه با آن کار کردید، مثال خیلی خوبی از method overloading است؛ شما می‌توانید به `println()` هر ورودی دلخواهی بدهید و آن به نحوی یک نمایش متنی از آن ورودی را برای شما چاپ می‌کند. در زبان‌هایی که method overloading ندارند، کار سخت‌تر می‌شود. مثلاً برای چاپ چیزهای مختلف باید متدهای مختلف با اسم‌های مختلف تعریف کنیم و در آن صورت، این مسئولیت روی دوش شما می‌افتد که متد درست را انتخاب کنید. بیایید یک مثال دیگر از method overloading ببینیم:

```
public class Sum {
    // Overloaded sum(). This sum takes two int parameters
    public int sum(int x, int y) {
        return (x + y);
    }

    // Overloaded sum(). This sum takes three int parameters
    public int sum(int x, int y, int z) {
        return (x + y + z);
    }

    // Overloaded sum(). This sum takes two double
    // parameters
    public double sum(double x, double y) {
        return (x + y);
    }

    public static void main(String[] args) {
        Sum s = new Sum();
        System.out.println(s.sum(10, 20));
        System.out.println(s.sum(10, 20, 30));
        System.out.println(s.sum(10.5, 20.5));
    }
```

```
}
```

همان‌طور که می‌بینید، اینجا سه تا متد با نام یکسان sum داریم، ولی ورودی‌های آن‌ها متفاوت است. هر سه‌تای این متدها دارند عمل جمع کردن را انجام می‌دهند، ولی یکی دو double را، یکی دو int را و یکی سه int را جمع می‌کند! به غیر از نوع ورودیها و تعدادشون، با تغییر دادن ترتیب ورودیها هم میشود متدها را overload کرد:

```
class Student {
    // Method 1
    public void getStudentInfo() {
        System.out.println("Name :" + name + " "
                + "ID :" + roll_ studentID);
    }

    // Method 2
    public void getStudentInfo (String name) {
        // Again printing name and id of person
        System.out.println("ID :" + studentID + " "
                + "Name :" + name);
    }
}
```

بعد از این که با مباحث مربوط به polymorphism و متدهای override شده آشنا شدید، دوباره به method overloading برخواهیم گشت.

۷.۱ Reference type ها

همان‌طور که تا الآن به خوبی می‌دانید، در جاوا، type ها به دو دسته‌ی primitive type و reference type تقسیم‌بندی می‌شوند. primitive type ها، تایپ‌های بسیار ساده‌ای مثل int ، char ، boolean و امثال آن‌ها هستند. فهرست کامل آن‌ها در لیست زیر آمده است:

Type	Definition	Approximate range or precision
boolean	Logical Value	true or false
char	16-bit, Unicode character	$64k$ characters
byte	8-bit, signed integer	-128 to 127
short	16-bit, signed integer	$-32,768$ to $32,676$
int	32-bit, signed integer	$-2.1e9$ to $2.1e9$
long	64-bit, signed integer	$-9.2e18$ to $9.2e18$
float	32-bit, IEEE 754, floating-point value	$6-7$ significant decimal places
double	64-bit, IEEE ۷۵۴	15 significant decimal places

هر تایپ دیگری در جاوا، reference type است. String ، JFrame ، ArrayList و حتی تایپ‌هایی مثل Car و Student که تا این‌جای کار تعریف کردیم، همگی reference type اند. هر reference type ای با یک کلاس تعریف شده است.

۱.۷.۱ تفاوت reference type ها و primitive type ها

همان‌طور که می‌دانید، تمام متغیرهای برنامه‌های شما، در حافظه‌ی خاصی به اسم RAM ذخیره می‌شوند. سیستم عامل، متغیرهای شما را در دو بخش متفاوت از این حافظه، به اسم stack و heap نگه می‌دارد. با این دو در درس‌های ساختمان داده و سیستم عامل بیشتر آشنا می‌شوید، ولی فعلاً بدانید که حافظه‌ی stack ، از heap سریع‌تر است، ولی در مقابل کمی کم‌حجم‌تر است.[1]

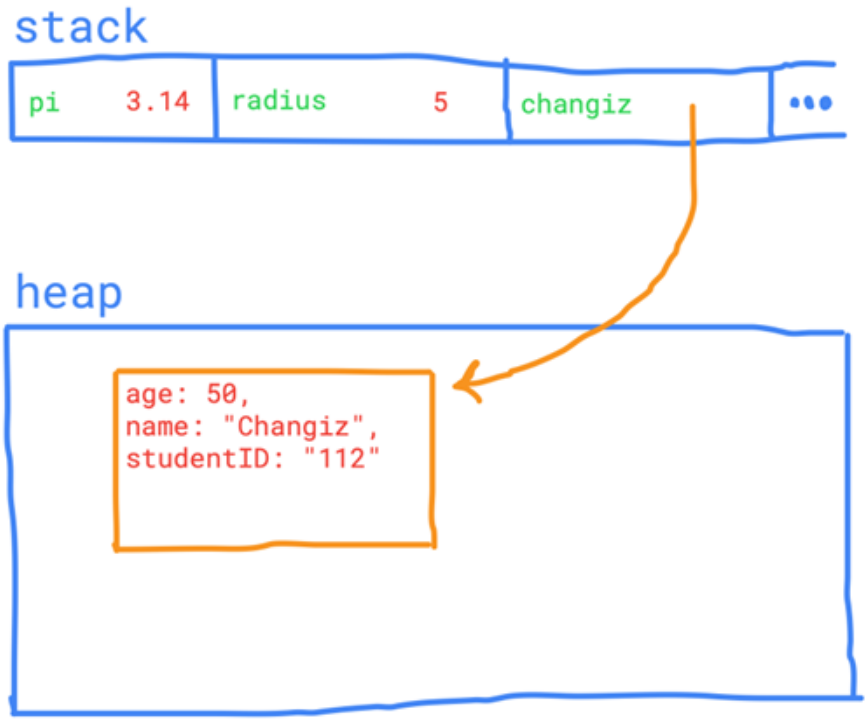
متغیرهایی که از جنس primitive type تعریف می‌کنید، حجم کمی دارند و بین 1 تا 8 بایت از حافظه را اشغال می‌کنند. به همین خاطر، جاوا آن‌ها را در stack نگه می‌دارد تا از سرعت بهتر stack استفاده کند و همزمان، stack سریع پر نشود. از طرفی، ممکن است object هایی که در برنامه‌ی خود تعریف می‌کنید -و همیشه از جنس reference type هستند-، حجم بسیار بیشتری داشته باشند. جاوا، اطلاعات این ها object را در heap ذخیره می‌کند و در stack ، صرفاً یک اشاره‌گر (pointer) به آن‌ها نگه می‌دارد.

مثلاً، برنامه‌ی زیر را در نظر بگیرید:

```
public class Main {
    public static void main(String[] args) {
        int radius = 5;
        double pi = 3.14;
        Student changiz = new Student(50, "Changiz", "112");
    }
}
```

اگر موری را حین اجرای این برنامه ببینیم، همچین شکلی دارد:

[1] البته، در واقعیت ممکن است اینگونه نباشد. Stack و Heap به خودی خود از دیگری سریع‌تر یا حجیم‌تر نیستند و ما در این‌جا خیلی ساده‌سازی می‌کنیم. در درس‌های بعدی خود بهتر می‌فهمید که تفاوت این دو با هم چیست. برای الآن، فرض کنید چیزی که گفتیم کاملاً درست است.



اگر مقدار خود متغیر چنگیز را چاپ کنید:

```
System.out.println(changiz);
```

همچین خروجی‌ای می‌بینید:

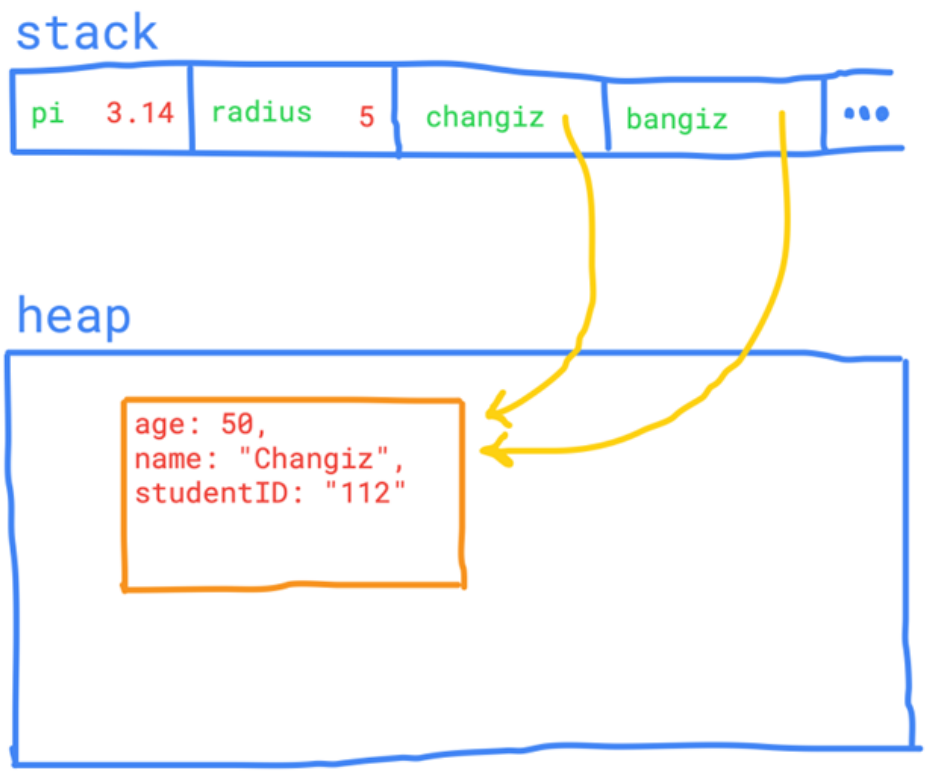
```
Student@6acbcfc0
```

متغیر changiz ، در واقع صرفا یک pointer یا reference به یک آبجکت از جنس Student است و نه چیزی بیشتر. با استفاده از اپراتور نقطه (.) ، می‌توانید به field ها و method های این آبجکت دسترسی داشته باشید.

این اتفاق، یک ساید افکت جالب روی کدهای شما دارد. فرض کنید، یک دانشجوی دیگر به اسم بنگیز درست کردیم و ان را مساوی با چنگیز قرار دادیم:

```
Student bangiz = changiz;
```

حالا، بنگیز و چنگیز هر دو به یک نقطه از heap اشاره می‌کنند:



حالا اگر شما، شماره دانشجویی چنگیز را عوض کنید:

```
changiz.studentID = "40113";
```

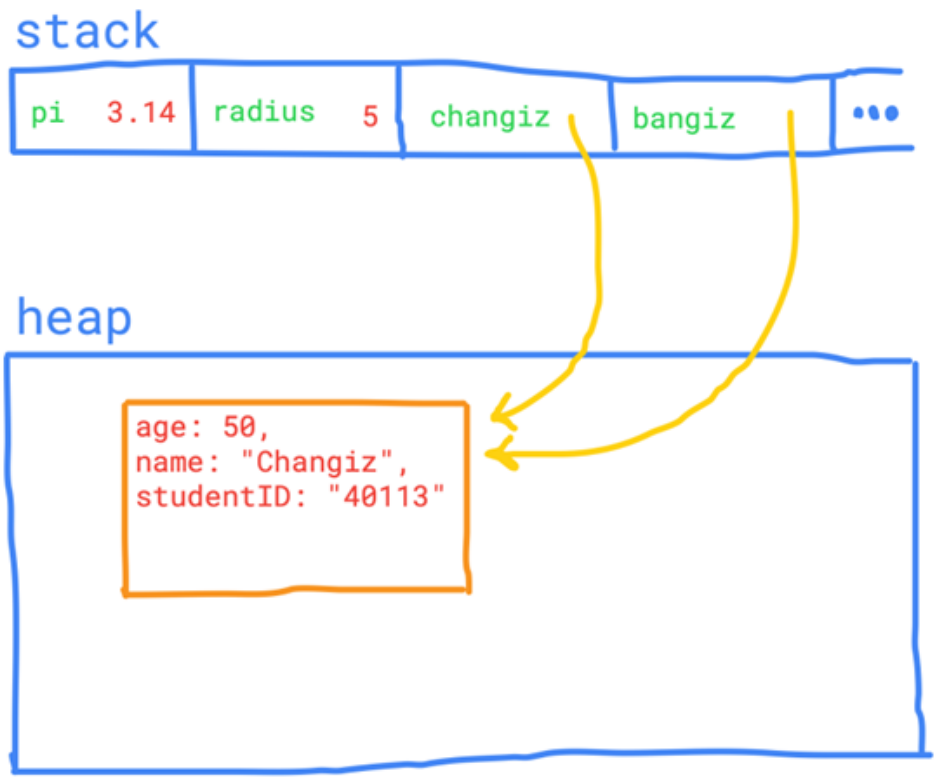
و بعد، شماره دانشجویی چنگیز و بنگیز را چاپ کنید:

```
System.out.println("Changiz studentID: " + changiz.studentID);
System.out.println("Bangiz studentID: " + bangiz.studentID);
```

می‌بینید که شماره دانشجویی بنگیز هم عوض شده است!

```
Changiz studentID: 40113
Bangiz studentID: 40113
```

عجیب نیست؟ اگر به موری نگاه کنید، بعد از عوض شدن شماره دانشجویی چنگیز، همچین وضعیتی دارد:



چنگیز، شماره دانشجویی آبجکتی که به ان اشاره می‌کرده را تغییر داده است. دست بر قضا، بنگیز هم به همین آبجکت اشاره می‌کرده و در نتیجه، شماره دانشجویی بنگیز هم واقعا عوض شده است. یک جای دیگر هم اتفاق مشابه‌ای می‌افتد. برای دیدن آن، متد زیر را به کدتان اضافه کنید:

```
public static void resetID(Student student) {
    student.studentID = "00000000";
}
```

این متد، یک Student را ورودی می‌گیرد، و شماره دانشجویی او را دستکاری می‌کند. حالا با استفاده از کد زیر، چنگیز را به این متد ورودی بدهید و بعد، شماره دانشجویی چنگیز و بنگیز را چاپ کنید:

```
resetID(changiz);

System.out.println("Changiz studentID: " + changiz.studentID);
System.out.println("Bangiz studentID: " + bangiz.studentID);
```

خروجی، به این شکل است:

```
Changiz studentID: 00000000
Bangiz studentID: 00000000
```

شاید بتوانید حدس بزنید که این‌جا چه اتفاقی افتاد. با ورودی دادن چنگیز به `resetID` ، در واقع شما اشاره‌گرتان به heap را به این تابع ورودی دادید. پس متغیر `student` در `resetID` و `changiz` و `bangiz` ، هر سه به یک نقطه از heap اشاره می‌کنند و مثل قبل، با تغییر فیلدهای یکی از آن‌ها، هر سه تغییر می‌کنند. به این نوع ورودی دادن به توابع، اصطلاحاً passing by reference می‌گویند.

مشابه هیچ کدام از این اتفاق‌ها، برای type primitive ها نمی‌افتد. چون همیشه در stack نگه‌داری می‌شوند و پوینتری به heap برای آن‌ها نگه‌داری نمی‌شود.

۲.۷.۱ کلاس‌های wrapper برای primitive هاtype

هر کدام از type primitive ها، تایپی مشابه از جنس reference type هم دارند. این تایپ‌ها، در جدول زیر آمده‌اند:

Primitive	Wrapper
`void`	`java.long.Void`
`boolean`	`java.lang.Boolean`
`char`	`java.lang.Character`
`byte`	`java.lang.Byte`
`short`	`java.lang.Short`
`int`	`java.lang.Integer`
`long`	`java.lang.Long`
`float`	`java.lang.Float`
`double`	`java.lang.Double`

نیازی نیست که خیلی درگیر این تایپ‌های جدید باشید، ولی بدانید که وجود دارند. وقتی که یک آرایه از int ها تعریف می‌کنید، از آن‌ها استفاده می‌کنید:

```
var arr = new ArrayList<Integer>();
```

اگر دقت کنید، به جای این که بین دو براکت از `int` استفاده کنید، از `Integer` استفاده کردید. اگر روی آن کلیک راست کنید و از Go To به Declaration and Usages بروید، می‌توانید ببینید که پشت آن یک کلاس است:

```
74  @jdk.internal.ValueBased
75  public final class Integer extends Number
76          implements Comparable<Integer>, Constable, ConstantDesc {

        A constant holding the minimum value an int can have, -2^31.

81      @Native public static final int MIN_VALUE = 0x80000000;
82

        A constant holding the maximum value an int can have, 2^31-1.

87      @Native public static final int MAX_VALUE = 0x7fffffff;
88

        The Class instance representing the primitive type int.

        Since: 1.1
```

این کلاس، معادل reference type گونه‌ای برای `int` است، و وقتی با کلاس‌ها سر و کار دارید از آن استفاده می‌شود. اگر خواستید بیشتر راجع به آن بدانید، به این داکیومنت رجوع کنید.

۸.۱ Garbage Collection

قبل از اینکه برویم سراغ Garbage Collection ، بیایید اول مفهوم نشت حافظه (Memory Leak) را بررسی کنیم.

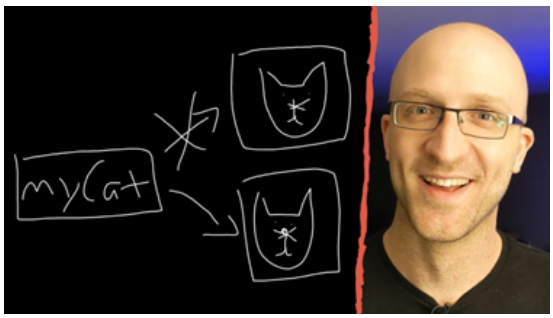
۱.۸.۱ آشنایی با Memory Leak

در بعضی از زبان‌های برنامه‌نویسی (مثل C و C++) مسئولیت «آزاد کردن» حافظه بر عهده خود برنامه‌نویس است. این یعنی شما باید هر وقت که دیگر به یک شی نیاز نداشتید، خودتان آن حافظه‌ی را آزاد کنید. مثلاً اگر با استفاده از تابع `malloc` مقداری از حافظه را تخصیص (allocate) دادید، وقتی که دیگر به این حافظه نیاز نداشتید، باید خودتان با استفاده از تابع `free` آن حافظه را آزاد کنید. بیایید یک مثال ببینیم:

```
#include <stdlib.h>

int main() {
    // Allocate memory dynamically
    int *ptr = (int *)malloc(sizeof(int) * 5);  // Allocating memory for 5
↪   integers

    // Use the allocated memory
    for (int i = 0; i < 5; i++) {
        ptr[i] = i + 1;
    }
    /*
     * Doing some stuff with these numbers
     */

    // Forgetting to free the allocated memory causes a Memory Leak
    // free(ptr); // If we uncomment this line, the Memory Leak will be
↪   avoided.

    return 0;
}
```

در این برنامه که به زبان (!) C نوشته شده، اول به اندازه 5 متغیر `int` حافظه اشغال می‌کنیم. اشاره‌گر `ptr` به اولین خانه از این 20 بایت [۲] اشاره می‌کند. حال می‌آییم 5 تا عدد صحیح را در حافظه ذخیره می‌کنیم. فرض کنید با این اعداد کارهایی انجام دادیم و الان کارمان با آن‌ها تمام شده است اما فراموش کردیم که این 20 بایت حافظه را آزاد کنیم! در حقیقت باید با صدا زدن تابع `free` اعلام می‌کردیم که ما دیگر به این 20 بایت نیازی نداریم تا این حافظه آزاد بشود.

[۲] با این فرض که هر متغیر از جنس `int` ، حافظه‌ای به اندازه 4 بایت را اشغال کند.



شکل ۱.۱: **Garbage Collection Explained – How It Saves your Lazy Java's Thumbnail: Programmer Bu**

اگر آزادش نکنید چه می‌شود؟ در این صورت آن حافظه همچنان در اختیار برنامه قرار دارد و به اصطلاح Memory Leak رخ می‌دهد. این یعنی حافظه‌ای که دیگر به کار نمی‌آید، همچنان در اختیار برنامه باقی می‌ماند و هیچ‌وقت آزاد نمی‌شود. این موضوع می‌تواند باعث شود که برنامه به مرور زمان حافظه‌ی زیادی مصرف کند و سیستم دچار مشکلاتی مثل کندی عملکرد یا حتی crash بشود.

۲.۸.۱ Garbage Collection: یک راه‌حل خوب

زمانی که داریم راجع به Garbage Collection در جاوا صحبت می‌کنیم، به زبان ساده یعنی جاوا خودش می‌رود و حافظه‌ای که دیگر به هیچ‌کار نمی‌آید را آزاد می‌کند. مثلاً وقتی که شما یک شی را در برنامه می‌سازید و دیگر به آن نیاز ندارید، جاوا خود به خود این شی را پاک می‌کند. شما اصلاً نیازی نیست که خودتان حافظه را آزاد کنید، همه چیز به صورت خودکار اتفاق می‌افتد!

شاید بپرسید که چطور این کار انجام می‌شود؟ خب، جاوا از تعدادی الگوریتم برای این کار استفاده می‌کند. ولی ما آن‌ها را اینجا بررسی نمی‌کنیم. در این ویدیوی یوتیوب و این داک اوراکل می‌توانید مطالب بیشتری در رابطه با این موضوع ببینید.

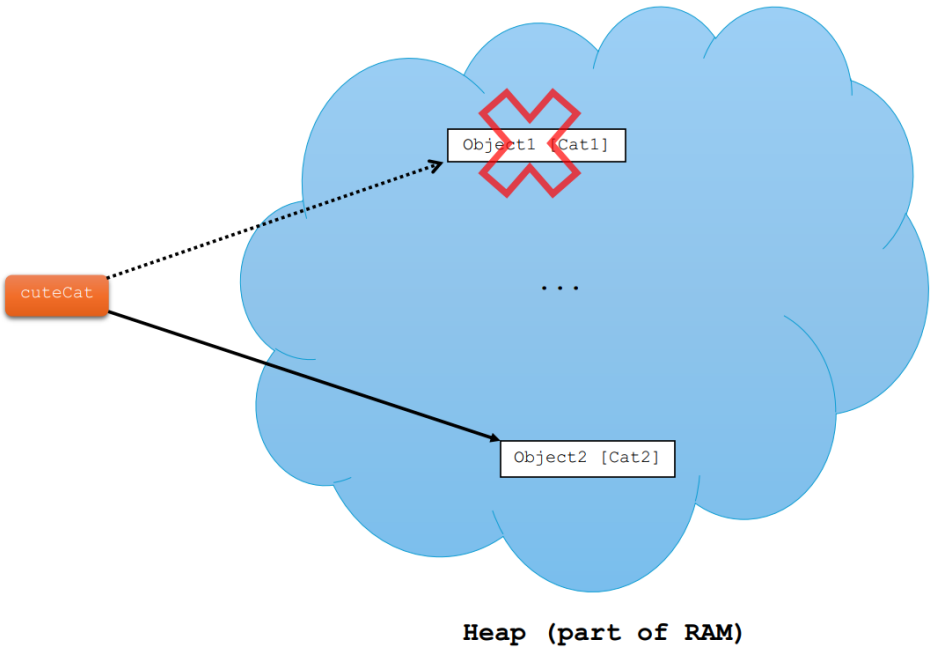
به طور کلی این ویژگی جاوا باعث می‌شود که شما تمرکز بیشتری روی منطق برنامه‌نویسی داشته باشید و دیگر نگران مدیریت دستی حافظه نباشید.

حالا در کد زیر، می‌توانید رفتار Garbage Collector را در ارتباط با آبجکت `cuteCat` بگویید؟

```
Cat cuteCat = new Cat("Cat 1");
cuteCat = new Cat("Cat 2");
```

شکل زیر میتواند نمایش خوبی از اتفاقات باشد. در اینجا `cuteCat` به جایی از حافظه اشاره [۳] می‌کند که در آن یک آبجکت از جنس `Cat` (که فیلد اسمش `Cat1` است) ذخیره شده. در خط بعدی `cuteCat` به جایی از حافظه اشاره می‌کند که در آن یک آبجکت از جنس `Cat` (که فیلد اسمش Cat2 است) ذخیره شده باشد. اما الان هیچ فلشی به `Cat1` وارد نمی‌شود، پس Garbage Collector آن را آزاد می‌کند.

[۳] دقت کنید که لفظ «اشاره کردن» در جاوا خیلی درست نیست. به خاطر این‌که ما در جاوا `pointer` نداریم و مدیریت حافظه را خود جاوا برایمان انجام می‌دهد. در جاوا حتی دسترسی مستقیم به حافظه هم نداریم (برخلاف C). اما برای مشابهت با زبان C ، در اینجا هم از لفظ اشاره کردن استفاده کردیم. در جاوا به متغیر هایی مثل `cuteCat` ، می گویند reference variable .



۳.۸.۱ Garbage Collector حواسش هست!

Garbage Collector آبجکت‌هایی که هنوز نیازشان داریم (بهشان رفرنس داریم) را پاک نمی‌کند. کد زیر را ببینیم:

```
class Duck {
    String name;
}

public class Main {
    public static void main(String[] args) {
        Duck duck = createDuck(); // a duck object will be created
        System.out.println(duck.name);
    }


    public static Duck createDuck() {
        Duck localDuck = new Duck();
        localDuck.name = "A White Duck";
        return localDuck;
    }
}
```

در این کد ابتدا یک کلاس خیلی ساده به نام Duck تعریف کردیم. متدی به نام createDuck نیز تعریف کردیم که ابتدا یک آبجکت از جنس Duck می‌سازد، سپس متغیر name را در این آبجکت

مقداردهی می‌کند و در نهایت این آبجکت را به عنوان خروجی متد برمی‌گرداند. بعد در متد main ، سعی می‌کنیم به متغیر name در این آبجکت دسترسی داشته باشیم.

اما نکته کجاست؟ احتمالاً در درس‌های قبلیتان خواندید که «متغیرهایی که در یک تابع تعریف می‌شوند عمرشان به اندازه‌ی اجرای همان تابع است و بعد از اتمام اجرای تابع، آن متغیر هم از بین می‌رود.» پس شاید انتظار داشته باشیم Garbage Collector آبجکت Duck را از بین ببرد! اما واقعیت آن است که Garbage Collector حواسش است که ما به کدام آبجکت‌ها هنوز نیاز داریم و نباید پاکشان کند. اینجا هم چون Duck را به عنوان خروجی برگرداندیم، یعنی آن را نیاز داریم، پس پاکش نمی‌کند.

خروجی کد بالا به صورت زیر است:

```
A White Duck
```

۴.۸.۱ یک نکته در مورد کلاس‌ها

در هر فایل جاوا (فایل با پسوند .java)، می‌توانیم حداکثر یک کلاس public داشته باشیم، همچنین اسم این کلاس public باید حتما با اسم فایل یکی باشد. مثال درست زیر را ببینید:

```java
// MyClass.java
public class MyClass {
    public void sayHello() {
        System.out.println("Hello, Java!");
    }
}
```

کدهای زیر نادرست هستند:

```java
// MyFile.java
public class MyClass { } // Error! The class name does not match the file
↪  name.
```

```java
// MyFile.java
public class MyClass { }

public class AnotherClass { } // Error! Only one public class is allowed.
```

۹.۱ Packages

۱.۹.۱ نیاز به منظم کردن فایل‌ها

در برنامه‌هایی که در جاوا می‌نویسیم، همیشه از کلاس ها یا اینترفیس ها [۴] استفاده می کنیم. برنامه‌ی ساده زیر که در کلاس Sample نوشته شده را ببینید:

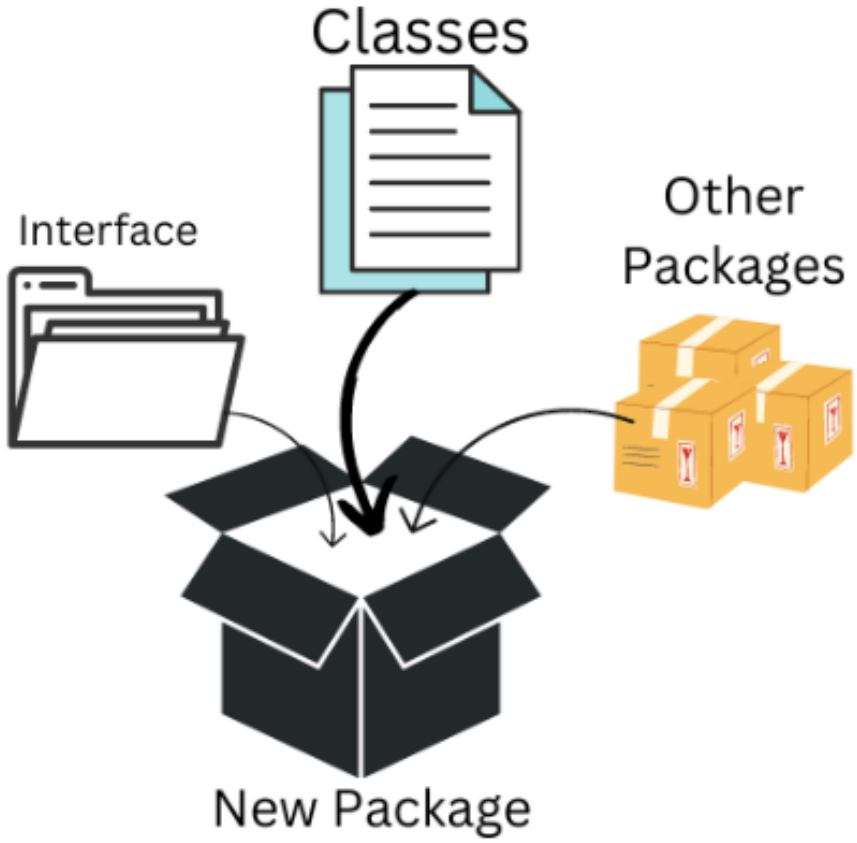
```
public class Sample {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

ولی برنامه‌های پیچیده‌تر ممکن است از صدها کلاس تشکیل شده باشند. اگر همه‌ی این کلاس‌ها را بدون هیچ نظمی کنار هم قرار بدهیم باعث می‌شود برنامه‌مان ناخوانا باشد و خودمان هم گیج می‌شویم.
احتمالاً یکی از اولین چیزهایی که برای منظم کردن فایل‌ها به ذهن ما می‌رسد استفاده از پوشه‌هاست. و این دقیقاً همان امکانی است که جاوا برای منظم کردن کلاس‌های برنامه‌مان برای ما فراهم کرده: ایجاد packageهای مختلف.

۲.۹.۱ پکیج چیست؟

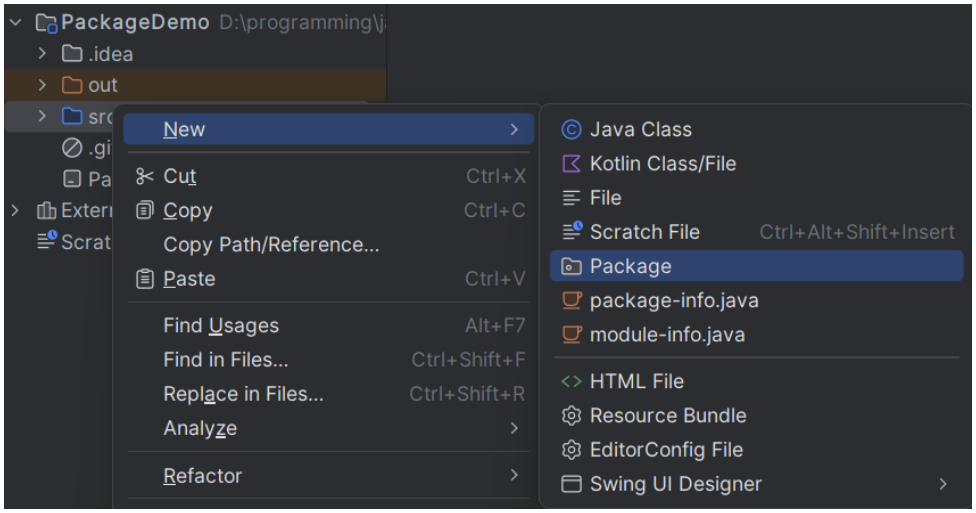
پکیج‌ها در جاوا مثل پوشه‌هایی هستند که کدهای برنامه‌نویسی (شامل کلاس‌ها و اینترفیس‌ها) را در آن‌ها قرار می‌دهیم تا همه چیز منظم و مرتب باشد. وقتی که برنامه‌های پیچیده‌تر را می‌نویسیم، تعداد کلاس‌ها زیاد می‌شود و اینجاست که پکیج‌ها به کمکمان می‌آیند تا بتوانیم این کلاس‌ها را دسته‌بندی کنیم.

[۴] با اینترفیس‌ها بعداً آشنا می‌شوید. فعلاً کاری به آن‌ها نداریم.

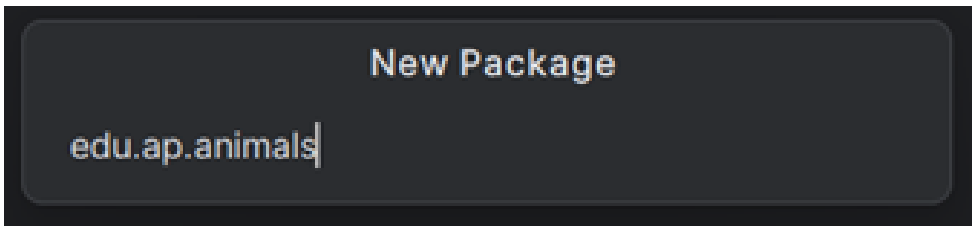


۳.۹.۱ چطور یک پکیج ایجاد کنیم؟

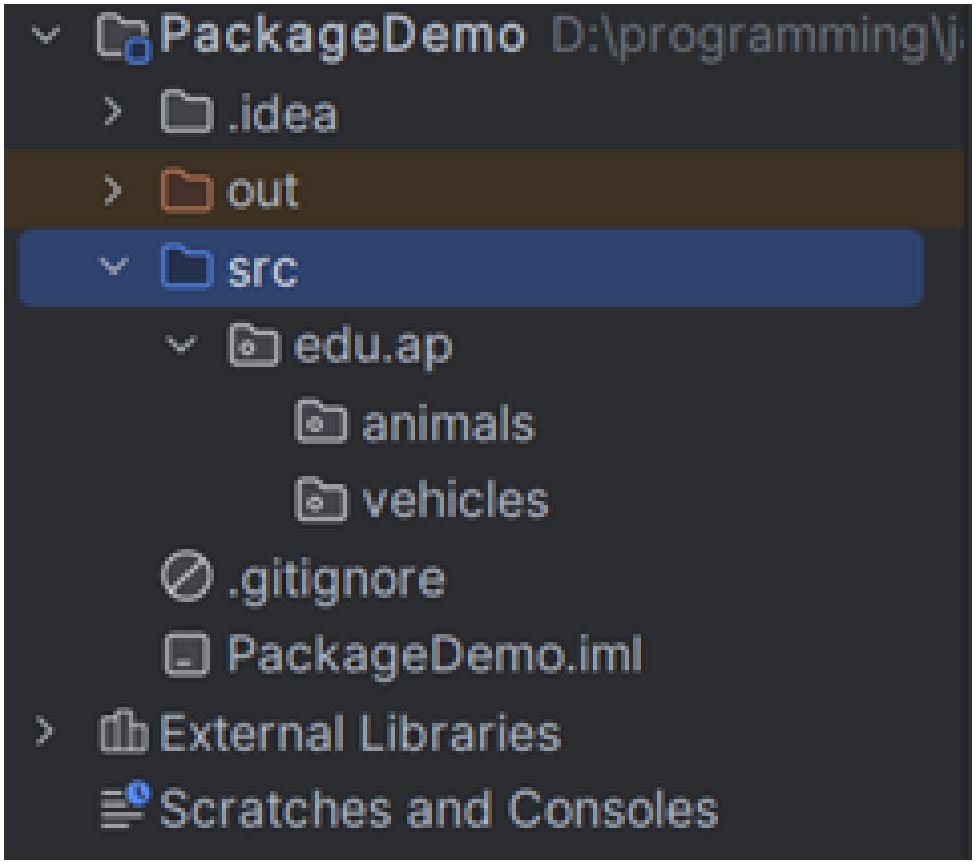
اول یک پروژه به اسم PackageDemo ایجاد کنید. حال روی پوشه‌ی src راست کلیک کنید و از نوار New ، گزینه‌ی Package را انتخاب کنید.



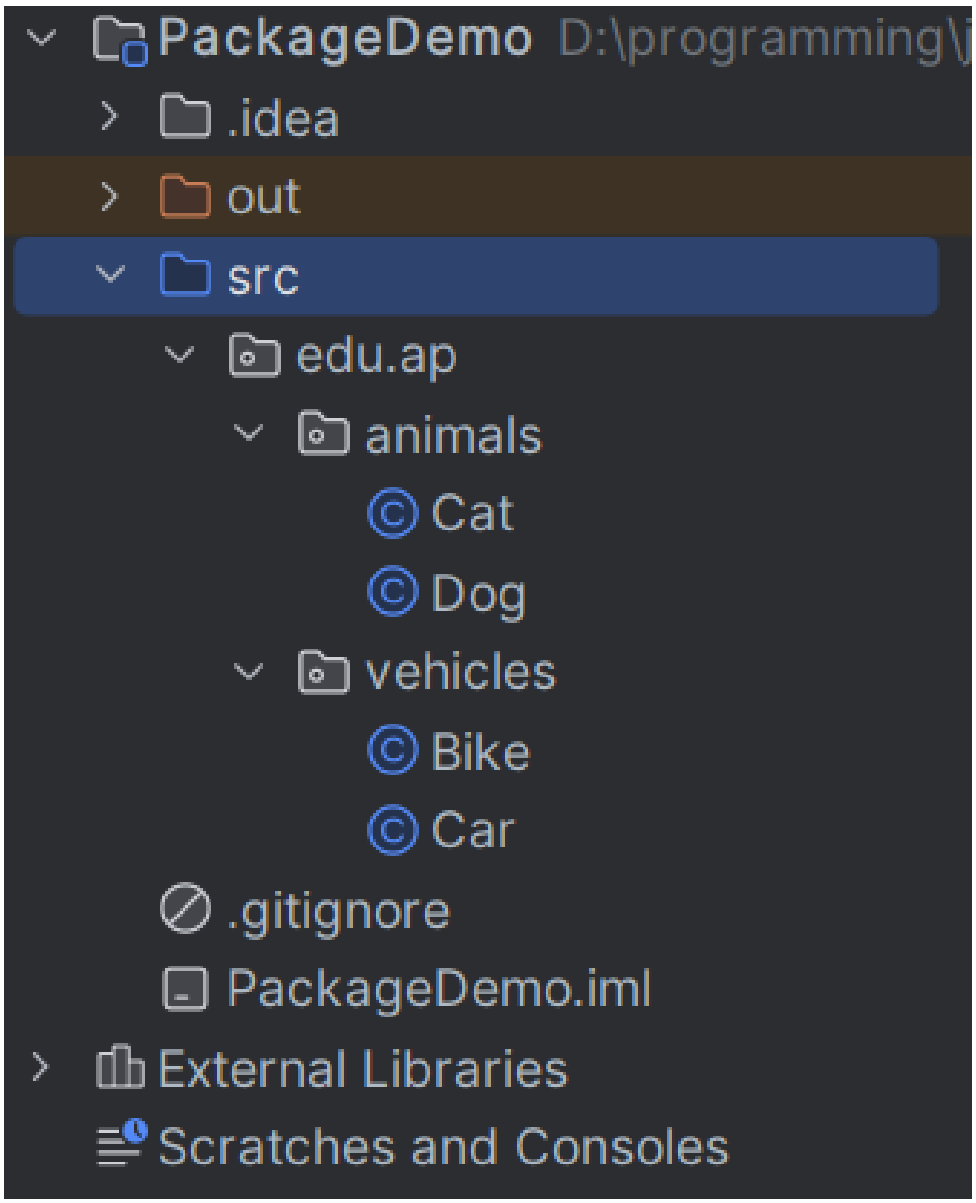
حالا اسم پکیج را وارد کنید. مثلا در اینجا edu.ap.animals :



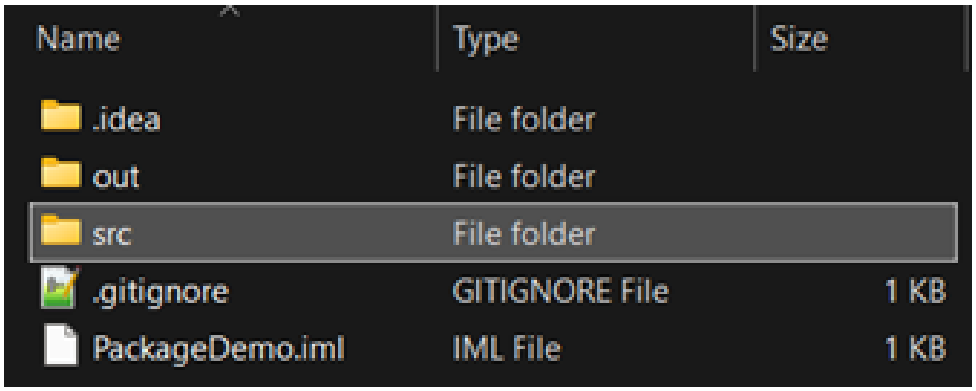
به همین شکل پکیج `edu.ap.vehicles` را هم بسازید.
الآن باید چیزی شبیه به تصویر زیر را در پوشه‌ی `src` داشته باشید:

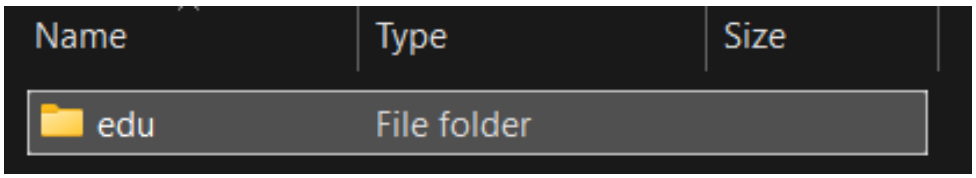


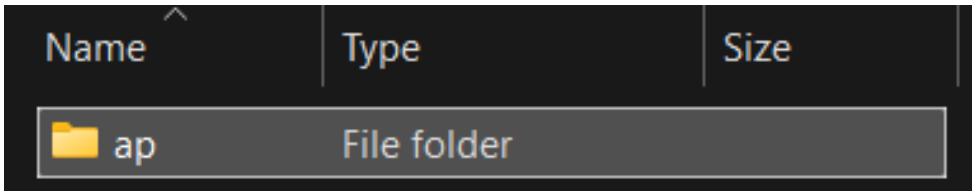
حال می‌خواهیم چند کلاس به هر کدام از این پکیج‌ها اضافه کنیم. مثلاً کلاس‌های `Dog` و `Cat` را به پکیج `animals` و کلاس‌های `Bike` و `Car` را به پکیج `vehicles` اضافه کنیم. در نهایت باید چیزی شبیه به تصویر زیر را داشته باشید:

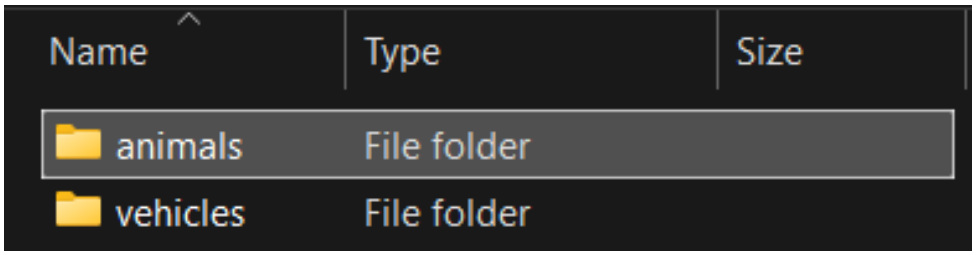


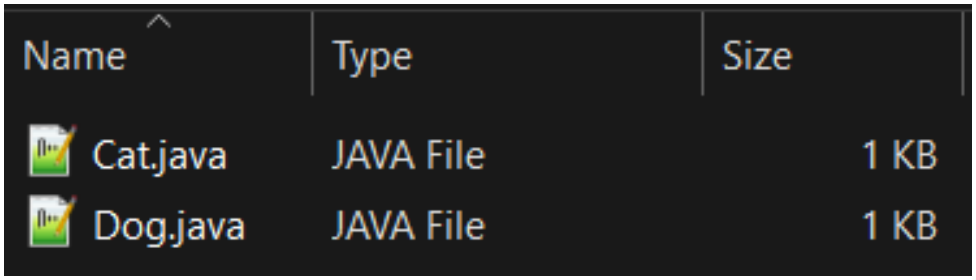
پکیج‌هایمان را ساختیم! حالا ببینیم واقعا چه فایل‌هایی ایجاد شده است. اگر از توی explorer به محل ایجاد پروژه‌تان بروید و به ترتیب وارد پوشه‌های `src`، `edu`، `ap` و `animals` بشوید چیزی شبیه به این‌ها می‌بینید:



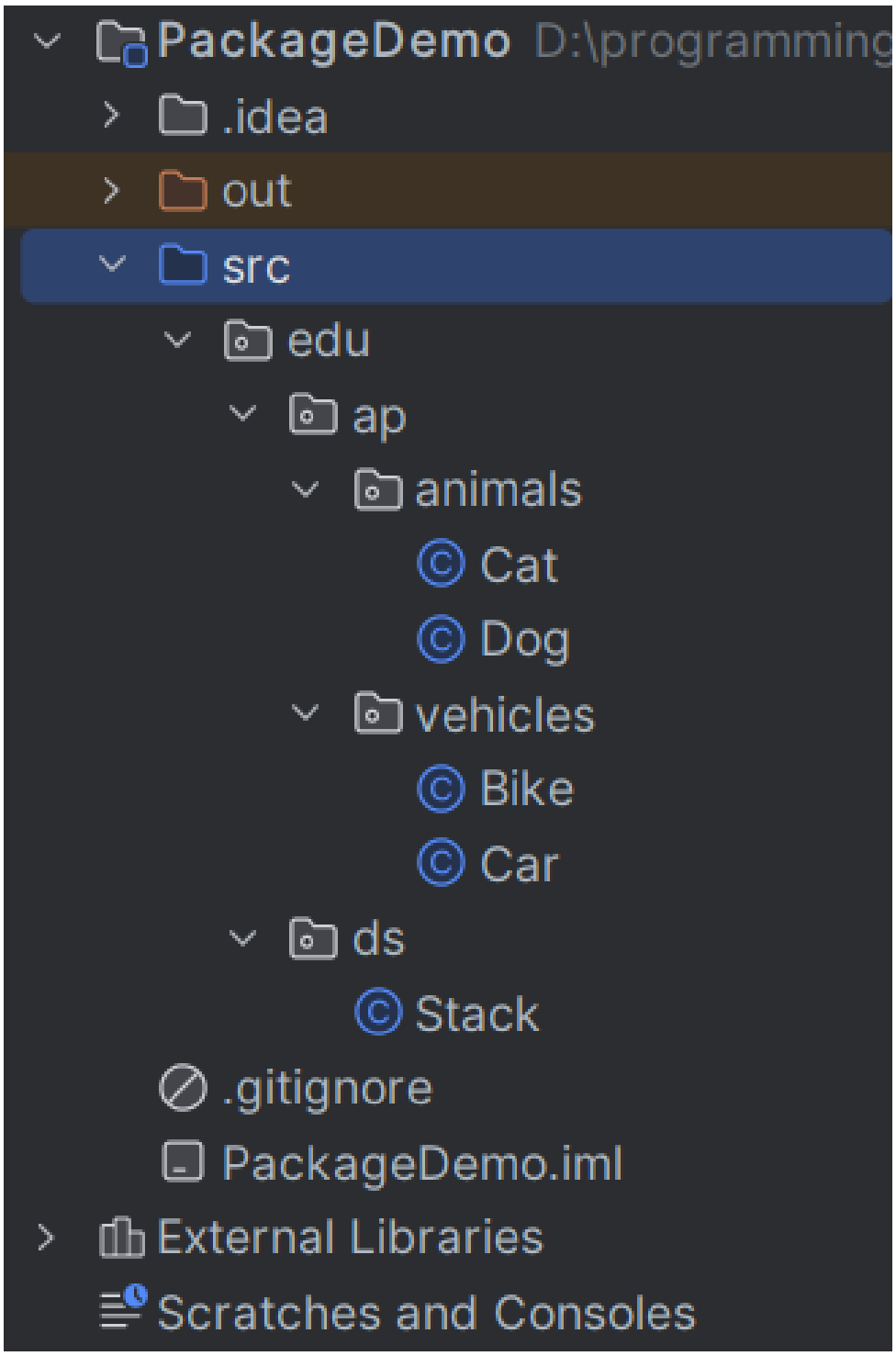








پس دیدیم که وقتی ما داریم یک پکیج تعریف می‌کنیم، واقعا پوشه ایجاد می‌شود!
نهایتاً پکیج `edu.ds` شامل کلاس `Stack` را درست کنید. در نهایت باید چیزی شبیه به این ببینید:



الآن برویم کد Cat.java را ببینیم:

```
package edu.ap.animals;

public class Cat {
}
```

همان‌طورکه می‌بینید، عبارت `package edu.ap.animals;` به ابتدای این کد اضافه شده است. این خط را نباید پاک کنید، زیرا در آن صورت جاوا متوجه نمی‌شود `Cat` متعلق به کدام پکیج است و خطای کامپایل دریافت خواهید کرد.

۴.۹.۱ نام‌گذاری متداول پکیج‌ها (Naming Conventions)

۱. پکیج‌ها همواره با حروف کوچک نام‌گذاری می‌شوند. مثلاً `java.util`

۲. از کلمات رزرو شده‌ی جاوا استفاده نکنید. کلماتی مثل `class`، `public`، `static` و...

۳. از زیرخط یا underscore _ ، خط‌تیره یا dash - ، فاصله یا space و کاراکترهای خاص (مثل @ ، $ ، &) استفاده نکنید. در نام‌گذاری پکیج‌ها فقط مجازیم از) dot نقطه .) استفاده کنیم.

   ۱. نادرست: `com.github.my-awesome-project`

   ۲. درست: `com.github.myawesomeproject`

۴. یک قاعده‌ی دیگر در نام‌گذاری پکیج‌ها، reverse domain name است (برعکس نوشتن نام دامنه). اگر یک پروژه‌ای متعلق به شرکت یا سازمانی است که دامنه (domain) خودش را دارد، آن دامنه را به شکل برعکس می‌نویسیم. مثلاً شرکت Mozilla (که نام دامنه‌اش `mozilla.org` است) چند پکیج در جاوا دارد. یکی از این پکیج‌ها اسمش `org.mozilla.javascript` است (البته این قاعده در مورد پکیج‌های استاندارد خود جاوا صدق نمی‌کند).

۵.۹.۱ کلیدواژه‌ی Import

یک مثال عملی

بیایید در کلاس `Bike` یک آبجکت از کلاس `Cat` بسازیم (دقت کنید که این دو کلاس متعلق به دو پکیج متفاوت هستند):

```
package edu.ap.vehicles;

public class Bike {
    public static void main(String[] args) {
        Cat cat = new Cat();
    }
}
```

اگر سعی کنیم این کد را اجرا کنیم، موقع کامپایل به مشکل می‌خوریم:

```
java: cannot find symbol
symbol:   class Cat
  location: class edu.ap.vehicles.Bike
```

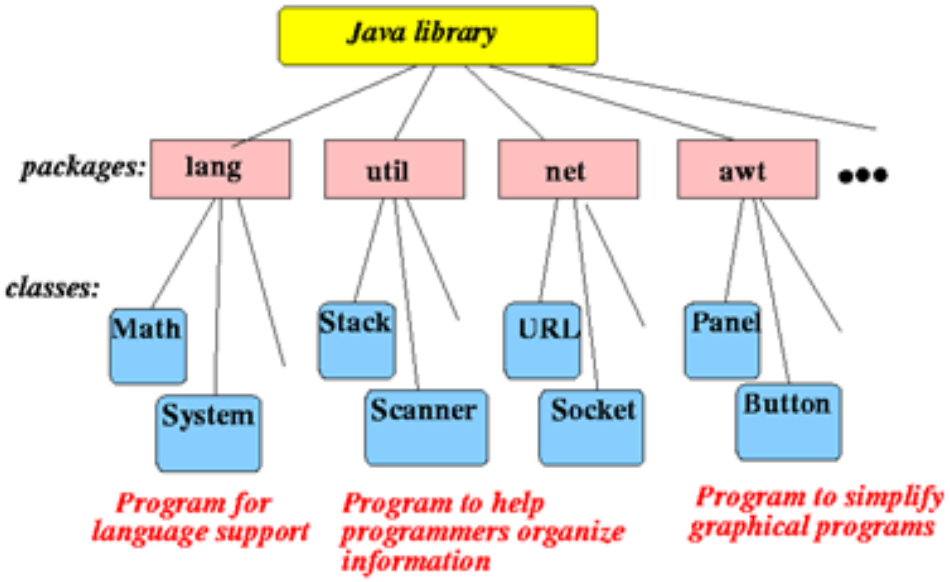
مشکل چیست ؟ جاوا نمیتواند کلاسی به اسم Cat را پیدا کند! دلیلش هم این است که جاوا فقط کلاس‌هایی را می‌بیند که توی همین پکیج هستند.
پس باید به نحوی کلاس Cat را به کدمان اضافه کنیم. کلیدواژه‌ی import دقیقا برای همین کار است. با استفاده از این کلیدواژه، ما به جاوا اعلام می‌کنیم که می‌خواهیم این کلاس را به کدمان اضافه کنیم:

```
package edu.ap.vehicles;
import edu.ap.animals.Cat;

public class Bike {
    public static void main(String[] args) {
        Cat cat = new Cat();
    }
}
```

اکنون دیگر کدمان کار می‌کند و می‌توانیم یک آبجکت از کلاس Cat بسازیم.

۶.۹.۱ در اهمیت پکیج‌ها

یکی از بزرگترین مزایای جاوا داشتن کتابخانه (library) های بسیار متنوع و کاربردی است.
می‌خواهید یک PDF درست کنید؟ کتابخانه مربوط به آن را import کنید. می‌خواهید با دیتابیس کار کنید؟ کتابخانه مربوط به آن را import کنید ...
مثلاً یکی از پکیج‌های مهم جاوا، java.lang است. این پکیج شامل کلاس‌های پایه‌ای مثل System ، Integer ، Math و String است.
در شکل زیر می‌توانید یک نمای کلی از پکیج‌های جاوا ببینید:



۷.۹.۱ import کردن کل پکیج

شما می‌توانید تمام کلاس‌های موجود در یک پکیج را یک‌جا import کنید. این کار را با عبارت * می‌توانید انجام بدهید. مثلاً کد زیر تمام کلاس‌های موجود در پکیج edu.ap.animals را import می‌کند.

```
import edu.ap.animals.*;
```

این قابلیت کار ما را خیلی اوقات آسان می‌کند. ولی انجام این کار همیشه هم مناسب نیست. import کردن دقیق کلاس‌ها علاوه بر خوانایی بیشتر کدمان، کمی هم زمان کامپایلمان را کمتر می‌کند.

دقت کنید که پکیج‌ها خودشان می‌توانند شامل پکیج باشند؛ همان‌طوری که پوشه‌ها می‌توانند داخل خود پوشه داشته باشند. اما عبارت * فقط _کلاس_های متعلق به پکیج را import می‌کند و sub-package ها را نمی‌کند (import کردن recursive نداریم).

مثلاً در نظر بگیرید که پکیج java.awt یک کتابخانه‌ی استاندارد جاوا است که شامل یک sub-package مثل java.awt.event است. کلاس Color متعلق به پکیج java.awt و کلاس ActionEvent متعلق به زیرپکیج java.awt.event است.

حال شما اگر به هر دوی این کلاس‌ها نیاز دارید باید هر کدام را جدا import کنید. کد زیر اشتباه است:

```
import java.awt.*;
```

دلیل اشتباه بودنش آن است که در نتیجه‌ی این کد کلاس Color ، ایمپورت می‌شود ولی کلاس ActionEvent نه.

کد زیر درست است:

```
import java.awt.Color;
import java.awt.event.ActionEvent;
```

دو مثال دیگر: مرور خاطرات

java.util.Scanner احتمالا یکی از اولین برنامه‌هایی که در جاوا نوشتید گرفتن ورودی از کاربر بوده است. مثلاً کد ساده‌ی زیر را ببینید:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        int x;
```

```
        Scanner scanner = new Scanner(System.in);
        x = scanner.nextInt();
    }
}
```

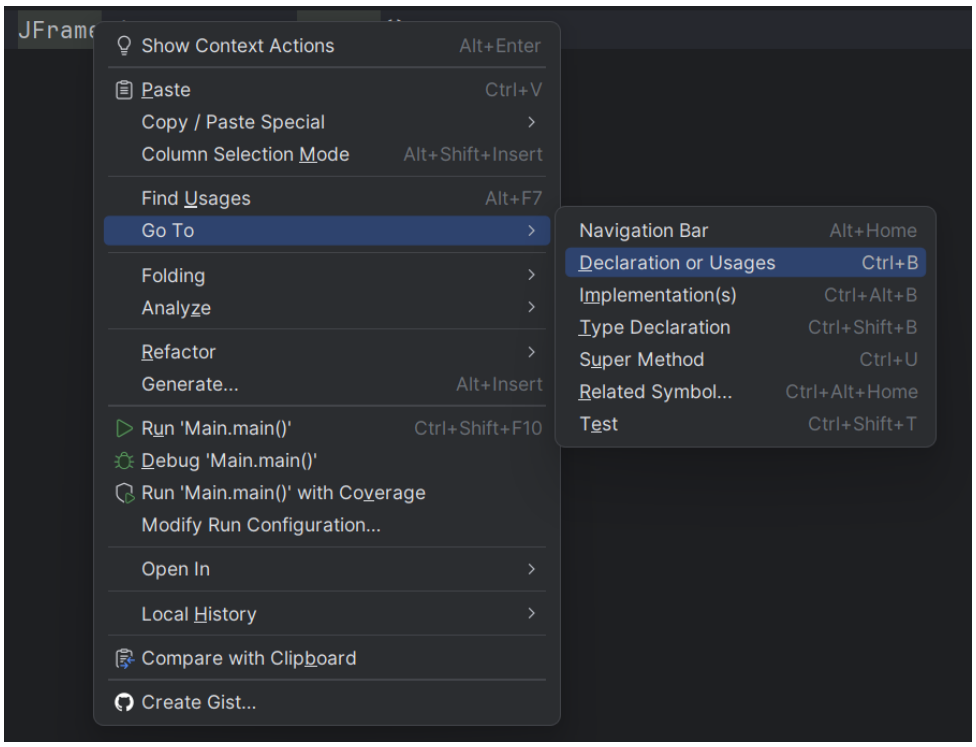
شاید قبلاً برایتان سوال شده باشد که آن import در خط اول چه کار می‌کند. اکنون احتمالاً می‌توانیم به راحتی به این سوال جواب بدهیم. در واقع کلاس Scanner متعلق به پکیج java.util است و ما چون می‌خواهیم از این کلاس در کدمان استفاده کنیم، آن را import کردیم.
حتی اگر وارد سورس کد این کلاس شوید، عبارت زیر را در خطوط ابتدایی می‌بینید:

```
package java.util;
```

javax.swing.JFrame در صورتی که با گرافیک کار کرده باشید، می‌دانید وقتی می‌خواستیم یک frame ایجاد کنیم، از کد زیر استفاده می‌کردیم:

```
import javax.swing.JFrame;

public class Main {
    public static void main(String[] args) {
        JFrame jFrame = new JFrame();
    }
}
```

همان‌طور که می‌توانیم حدس بزنیم، کلاس JFrame متعلق به پکیج javax.swing است و چون ما می‌خواهیم از این کلاس در کدمان استفاده کنیم، آن را import کردیم.
می‌توانیم خیلی راحت این را بررسی کنیم. هر موقع خواستید کد یک کلاس را ببینید (سورس کد جاوا)، می‌توانید روی اسم آن کلاس راست کلیک کنید و از نوار Go To قسمت Declaration or Usages را انتخاب کنید. می‌بینید که به سورس کد آن کلاس منتقل خواهید شد.



اگر این کار را انجام دهید، در فایل `JFrame.java` که به آن منتقل شده‌اید در خطوط ابتدایی عبارت `package javax.swing;` را می‌بینید.

۸.۹.۱ چند نکته در مورد پکیج‌ها

هر کلاس دقیقاً به یک پکیج تعلق دارد

هر کلاس دقیقاً به یک پکیج تعلق دارد. اگر کلاسی که نوشته‌اید را در یک پکیج قرار ندادید، به صورت پیش‌فرض این کلاس متعلق به default package خواهد بود. مثلاً کلاس زیر را در نظر بگیرید:

```
public class Student {
    private String name;
    private int age;

    // Some other variables and methods
}
```

دقت کنید که اکنون کلاس `Student` را نمی‌شود در پروژه‌ها یا کلاس‌های دیگر `import` و استفاده کرد؛ چون به پکیج نام‌گذاری شده‌ای تعلق ندارد (default package واقعاً اسم یک پکیج نیست).

تعریف نکردن پکیج برای کلاس‌هایمان در پروژه‌های کوچک ایرادی ندارد؛ ولی در پروژه‌های بزرگ باید پکیج‌های مناسبی ایجاد کنیم.

همچنین یک کلاس نمی‌تواند به بیش از یک پکیج تعلق داشته باشد (در غیر این صورت خطای کامپایل دریافت می‌کنیم).

استفاده از یک کلاس بدون import کردن آن

گاهی اوقات که فقط یک بار می‌خواهید از یک کلاس در کدتان استفاده کنید، می‌توانید آن کلاس را به طور مستقیم import نکنید و به جای آن به طور کامل به اسم پکیج در کد اشاره کنید. مثال زیر را ببینید:

```
public class Main {
    public static void main(String[] args) {
        javax.swing.JFrame jFrame = new javax.swing.JFrame();
    }
}
```

در اینجا ما بدون استفاده از عبارت `import javax.swing.JFrame;` توانستیم از کلاس `JFrame` استفاده کنیم. این‌گونه نوشتن شاید کمی طولانی به نظر بیاید، ولی در کلاس‌هایی که تعداد زیادی `import` دارند و یا قرار باشد کلاس‌هایی با نام یکسان `import` بشوند، به ما کمک می‌کند این مشکلات را حل کنیم.

۱۰.۱ چه چیزی یاد گرفتیم؟

در این داکیومنت، با مفاهیم پایه‌ای شی‌گرایی در جاوا آشنا شدیم و قدم اول برای نوشتن برنامه‌های ساخت‌یافته‌تر را برداشتیم. مهم‌ترین نکاتی که یاد گرفتیم:

- تفاوت بین **primitive type** و **reference type** و نحوه‌ی نگه‌داری آن‌ها در حافظه
- این که چگونه می‌توان با استفاده از `class` ، **type های جدید** تعریف کرد
- مفهوم **object (instance)** و نحوهٔ ساخت آن با استفاده از `new`
- آشنایی با **field** ها و نقش آن‌ها در نگه‌داری وضعیت یک شی
- تعریف و استفاده از **method** ها برای پیاده‌سازی رفتار کلاس‌ها
- نوشتن و استفاده از **constructor** ها برای مقداردهی اولیه‌ی object ها
- تفاوت بین **متغیرهای محلی** و **field ها**
- مفهوم **static** و تفاوت اعضای استاتیک با اعضای معمولی
- آشنایی با **this** و کاربرد آن در جلوگیری از shadowing
- مفهوم **method overloading** و استفاده از چند متد هم‌نام با ورودی‌های متفاوت
- درک بهتر از نحوه‌ی کار **reference ها** و تاثیر آن‌ها در تغییر داده‌ها
- آشنایی مقدماتی با **Garbage Collection** و مدیریت خودکار حافظه در جاوا

- اهمیت **package**ها برای سازمان‌دهی کد و استفاده از `import`

در ادامه‌ی مسیر، روی همین مفاهیم پایه، مباحث مهم‌تری مثل **کپسوله‌سازی (Encapsulation)** ، **وراثت (Inheritance)** و **چندریختی (Polymorphism)** را یاد خواهیم گرفت.